

ISPGAYA

instituto superior politécnico

Escola Superior de Ciência e Tecnologia

LEI

2025/2026

Unidade Curricular de 2º ano

LPP e ATW

Alunos:

Francisco Gouveia Veloso Vilela (2023103067)

Miguel Oliveira (2023103434)

Guilherme Guastaferrro (2023106093)

Docente: José Arnauld e Rui Silva

Vila Nova de Gaia, 02/06/2026

ÍNDICE DE CONTEÚDOS

Índice de Imagens.....	4
Índice de Tabelas.....	5
Abreviaturas.....	6
Resumo.....	9
Abstract.....	10
1. INTRODUÇÃO.....	11
1.1 Enquadramento e Relevância do Projeto.....	11
1.2 Objetivos do Projeto.....	12
1.2.1 Objetivos Pedagógicos.....	12
1.2.2 Objetivos Operacionais e Funcionais.....	12
1.3 Estrutura do Documento.....	13
2. ANÁLISE E ESPECIFICAÇÃO DE REQUISITOS.....	14
2.1 Descrição Geral do Sistema.....	14
2.2 Levantamento de Requisitos do Sistema.....	15
2.2.1 Requisitos Funcionais (RF).....	15
2.2.2 Requisitos Não Funcionais (RNF).....	17
2.3 Atores do Sistema e Casos de Uso.....	18
3. CONCEÇÃO DE INTERFACE E DESIGN (UI/UX).....	20
3.1 Identidade Institucional e Design System.....	20
3.2 Planeamento Visual e Protótipos (Wireframes).....	21
3.2.1 Homepage do Laboratório Cultural.....	21
3.2.2 Módulo de Teatro: Bilheteira e Mapa de Lugares.....	22
3.2.3 Biblioteca: Catálogo Virtual e Empréstimos.....	23
3.3 Experiência do Utilizador (UX) e Micro-Interações.....	24
4. ENGENHARIA DE DADOS E ARQUITETURA DE SOFTWARE.....	25
4.1 Arquitetura de Software (MVC e Blueprints).....	25
4.2 Estrutura e Engenharia da Base de Dados.....	27
4.2.1 Justificação das Decisões Técnicas de Armazenamento.....	27
4.2.2 Dicionário de Dados e Tabelas Relacionais.....	28
4.2.3 Integridade Referencial e Cardinalidades.....	29
5. LINGUAGENS E PARADIGMAS DE PROGRAMAÇÃO (LPP).....	30
5.1 Paradigma Orientado a Objetos (POO).....	30
5.2 Paradigma Funcional.....	31
5.3 Paradigma Imperativo e Procedimental.....	32
6. PADRÕES DE DESENHO DE SOFTWARE (DESIGN PATTERNS).....	34
6.1 Padrão Model-View-Controller (MVC).....	34
6.2 Padrão Application Factory.....	35
6.3 Padrão Decorator.....	35
6.4 Padrão Repository e Abstração de Acesso a Dados.....	36
7. TECNOLOGIAS E IMPLEMENTAÇÃO WEB (ATW).....	37
7.1 Implementação do Front-end.....	37
7.1.1 Estruturação HTML5 e Estilização Tailwind CSS.....	37

7.1.2 Interatividade Assíncrona com Vanilla JavaScript (Fetch API).....	37
7.2 Implementação do Back-end.....	39
7.2.1 Rotas Flask e Motor de Templates Jinja2.....	39
7.2.2 Integração de Dados e Operações CRUD no Back-office.....	39
7.3 Funcionalidades Opcionais e Inovadoras (Diferenciadores).....	40
7.3.1 Bilheteira de Teatro Interativa e Controlo de Concorrência.....	40
7.3.2 Geração Automática de Tickets PDF e Incorporação de QR Codes.....	41
7.3.3 Integração da API Google Gemini para o Assistente Virtual.....	41
7.3.4 Comunicação Automática por E-mail (Serviço SMTP).....	42
8. SEGURANÇA.....	42
8.1 Prevenção de Cross-Site Scripting (XSS).....	42
8.2 Prevenção de SQL Injection.....	43
8.3 Criptografia e Armazenamento Seguro de Passwords.....	44
8.4 Validação de Dados e Restrição de Uploads.....	45
8.5 Mecanismos de Avaliação de Robustez de Passwords.....	46
8.6 Engenharia de Sessões e Encerramento Seguro (Logout).....	46
8.7 Mapeamento de Riscos (Matriz OWASP Top 10).....	47
8.8 Segurança na Integração da API Google Gemini.....	49
9. TESTES E VALIDAÇÃO.....	50
9.1 Testes Funcionais.....	50
9.2 Testes Unitários e de Integração (pytest).....	55
9.3 Testes de Responsividade.....	59
9.4 Validação de Formulários.....	60
9.5 Compatibilidade de Motores de Renderização (Browsers).....	60
10. CONCLUSÃO.....	62
10.1 Balanço do Projeto e Cobertura de Requisitos.....	62
10.2 Métricas de Engenharia de Software.....	64
10.3 Dificuldades Encontradas e Resolução de Problemas.....	66
10.4 Trabalho Futuro.....	67
10.5 Visão Crítica e Oportunidades de Melhoria Arquitetónica.....	68
10.6 Reflexão Final e Aprendizagens.....	69
ANEXO A — Guião de Evidências Visuais de Testes Funcionais.....	70
Anexo 1: Autenticação Válida.....	70
Anexo 2: Proteção de Rotas.....	71
Anexo 3: Validação de Formulários.....	72
Anexo 4: Restrição de E-mail Académico.....	73
Anexo 5: Controlo de Duplicação.....	74
Anexo 6: Mapa de Sala.....	75
Anexo 7: Payload JSON da API.....	76
Anexo 8: Notificação SMTP.....	77

ÍNDICE DE IMAGENS

Fase de Design e Planeamento

Figura 1 - Wireframe da Homepage do Laboratório Cultural	pág. 20
Figura 2 - Wireframe da Bilheteira e Mapa de Lugares do Teatro	pág. 21
Figura 3 - Wireframe do Catálogo Virtual do Clube de Leitura	pág. 22

Fase de Arquitetura de Software

Figura 4 - Diagrama de Sequência do Fluxo de Requisição HTTP	pág. 25
Figura 5 - Diagrama Entidade-Relação (ERD) da Base de Dados	pág. 25

Fase de Testes Funcionais

Figura A1 - Painel de administração após autenticação bem-sucedida	pág. 69
Figura A2 - Bloqueio de acesso a rotas protegidas sem sessão	pág. 70
Figura A3 - Alertas de validação em campos obrigatórios não preenchidos ...	pág. 71
Figura A4 - Rejeição de formulário por submissão de e-mail não académico .	pág. 72
Figura A5 - Bloqueio de inscrição duplicada no módulo de teatro	pág. 73
Figura A6 - Renderização interativa do mapa de lugares do espetáculo	pág. 74
Figura A7 - Resposta estruturada em formato JSON fornecida pela API	pág. 75
Figura A8 - Receção de notificação automática via protocolo SMTP	pág. 76

Para a Página 4 (ÍNDICE DE TABELAS)

Tabela 1 - Requisitos Funcionais do Sistema	pág. 14
Tabela 2 - Requisitos Não Funcionais do Sistema	pág. 16
Tabela 3 - Matriz de Mapeamento OWASP Top 10 e Estratégias de Mitigação ..	pág. 46
Tabela 4 - Matriz de Testes Funcionais	pág. 50
Tabela 5 - Matriz de Cobertura de Requisitos e Estado Real	pág. 62
Tabela 6 - Métricas do Projeto	pág. 64

ÍNDICE DE TABELAS

Tabela 1 - Requisitos Funcionais do Sistema	pág. 14
Tabela 2 - Requisitos Não Funcionais do Sistema	pág. 16
Tabela 3 - Matriz de Mapeamento OWASP Top 10 e Estratégias de Mitigação .	pág. 46
Tabela 4 - Matriz de Testes Funcionais	pág. 50
Tabela 5 - Matriz de Cobertura de Requisitos e Estado Real	pág. 62
Tabela 6 - Métricas do Projeto	pág. 64

ABREVIATURAS

- DHCP - Dynamic Host Configuration Protocol
- DNS - Domain Name System
- IP - Internet Protocol
- LAN - Local Area Network
- VLAN - Virtual Local Area Network
- WAN - Wide Area Network
- RJ45 - Registered Jack 45 (conector de rede)
- HTTP - Hyper Text Transfer Protocol
- HTTPS - HyperText Transfer Protocol Secure

Outras abreviaturas e siglas técnicas definidas ao longo do texto:

- 3NF - Terceira Forma Normal
- AJAX - Asynchronous JavaScript and XML (chamadas assíncronas)
- API - Application Programming Interface
- ATW - Aplicações e Tecnologias Web
- CDN - Content Delivery Network
- CI/CD - Continuous Integration / Continuous Deployment (Integração Contínua e Entrega Contínua)
- CRUD - Create, Read, Update, Delete (Criar, Ler, Atualizar, Eliminar)
- CTAs - Call to Action (botões de ação)
- DOM - Document Object Model
- DoS - Denial of Service (Negação de Serviço)
- DRY - Don't Repeat Yourself

- E2E - End-to-End (testes de ponta a ponta)
- IA - Inteligência Artificial
- ISPGAYA - Instituto Superior Politécnico Gaya
- JSON - JavaScript Object Notation
- JWT - JSON Web Tokens
- LLMs - Large Language Models (Modelos de Linguagem de Grande Escala)
- LOC - Linhas de Código
- LPP - Linguagens e Paradigmas da Programação
- MVC - Model-View-Controller
- MVP - Minimum Viable Product (Produto Mínimo Viável)
- ORM - Object-Relational Mapper
- OWASP - Open Worldwide Application Security Project
- PoC - Proof of Concept (Prova de conceito)
- POO - Paradigma Orientado a Objetos
- PWA - Progressive Web App
- RBAC - Role-Based Access Control (Controlo de Acessos Baseado em Papéis)
- REST - Representational State Transfer (RESTful)
- RF - Requisitos Funcionais
- RNF - Requisitos Não Funcionais
- SEO - Search Engine Optimization
- SGBDR - Sistema de Gestão de Bases de Dados Relacionais
- SMTP - Simple Mail Transfer Protocol

- SQLi - SQL Injection (Injeção de código SQL)
- SSL/TLS - Secure Sockets Layer / Transport Layer Security
- UI - User Interface (Interface de utilizador)
- UX - User Experience (Experiência do Utilizador)
- XSS - Cross-Site Scripting

RESUMO

O presente projeto académico visa a conceptualização e o desenvolvimento da plataforma web "Laboratório Cultural" para o Instituto Superior Politécnico Gaya (ISPGAYA), integrando competências das unidades curriculares de Aplicações e Tecnologias Web e Linguagens e Paradigmas da Programação. A solução desenvolvida centraliza a divulgação e gestão cultural da instituição através de três eixos fundamentais: um Clube de Leitura com catálogo virtual e requisição automatizada; um Clube de Teatro suportado por um motor de bilheteira digital com mapa interativo de assentos, controlo de concorrência e emissão de bilhetes em formato PDF com código QR via SMTP; e a promoção da Tuna Académica. Do ponto de vista da engenharia de software, a arquitetura assenta na framework Flask (Python), aplicando rigorosamente paradigmas de programação (Orientado a Objetos, Funcional e Imperativo) e padrões de desenho (MVC, Application Factory, Repository e Decorator). A interface de utilizador (front-office) foi concebida com foco na ergonomia e responsividade, recorrendo a HTML5, Tailwind CSS e Vanilla JavaScript para interatividade assíncrona e fluida. O back-office disponibiliza um painel administrativo para a gestão de conteúdos, protegido por autenticação segura com encriptação bcrypt, prevenção de vulnerabilidades de injeção (XSS e SQLi) e Controlo de Acessos Baseado em Papéis (RBAC). Como elemento diferenciador, o sistema integra de forma segura um assistente virtual conversacional baseado na API Google Gemini, culminando numa plataforma robusta, escalável e inovadora que moderniza e democratiza o acesso à cultura no contexto do ensino superior.

Palavras-chave: Desenvolvimento Web, Engenharia de Software, Padrões de Desenho, Flask, Inteligência Artificial.

ABSTRACT

This academic project aims at the conceptualization and development of the "Cultural Laboratory" web platform for the Instituto Superior Politécnico Gaya (ISPGAYA), integrating competencies from the Web Applications and Technologies and the Programming Languages and Paradigms curricular units. The developed solution centralizes the institution's cultural dissemination and management through three fundamental axes: a Reading Club with a virtual catalog and automated requests; a Theater Club supported by a digital ticketing engine featuring an interactive seat map, concurrency control, and SMTP-based issuance of PDF tickets with QR codes; and the promotion of the Tuna Académica. From a software engineering perspective, the architecture is built upon the Flask framework (Python), rigorously applying programming paradigms (Object-Oriented, Functional, and Imperative) and design patterns (MVC, Application Factory, Repository, and Decorator). The user interface (front-office) was designed with a focus on ergonomics and responsiveness, utilizing HTML5, Tailwind CSS, and Vanilla JavaScript for seamless asynchronous interactivity. The back-office provides an administrative panel for content management, protected by secure authentication with bcrypt encryption, prevention of injection vulnerabilities (XSS and SQLi), and Role-Based Access Control (RBAC). As a differentiating element, the system securely integrates a conversational virtual assistant based on the Google Gemini API, culminating in a robust, scalable, and innovative platform that modernizes and democratizes access to culture within the higher education context.

Keywords: Web Development, Software Engineering, Design Patterns, Flask, Artificial Intelligence.

1. INTRODUÇÃO

1.1 ENQUADRAMENTO E RELEVÂNCIA DO PROJETO

A integração de atividades culturais no contexto do ensino superior desempenha um papel fulcral no desenvolvimento integral dos estudantes, complementando a formação técnico-científica com um enriquecimento humanístico e social indispensável. A promoção de espaços de criação, debate e fruição cultural fomenta não só o sentido de pertença à instituição, mas também o desenvolvimento de competências transversais, vulgarmente designadas por soft skills, que se revelam essenciais para a futura inserção no mercado de trabalho e para o exercício de uma cidadania ativa.

No atual contexto socioeconómico, vincadamente marcado por uma forte transição digital, a desmaterialização dos processos de divulgação e o acesso universal à cultura apresentam-se como um imperativo estratégico para as instituições de ensino. A centralização destas atividades numa plataforma digital dedicada otimiza os fluxos de comunicação, democratiza o acesso à informação e moderniza a interação entre os diversos intervenientes académicos, eliminando barreiras físicas, temporais e burocráticas na participação cultural.

Neste cenário de modernização tecnológica, surge o projeto da plataforma web "Laboratório Cultural", concebido especificamente para servir a comunidade académica do Instituto Superior Politécnico Gaya (ISPGAYA). Este portal web integrado visa consolidar, num único ecossistema digital, as diversas vertentes culturais promovidas pela instituição. Para tal, a plataforma agrega e dinamiza iniciativas estruturantes como o Clube de Leitura, o Clube de Teatro e a Tuna Académica, garantindo a estudantes, docentes e funcionários um canal unificado, interativo e eficiente para a vivência cultural no seio do instituto.

1.2 OBJETIVOS DO PROJETO

1.2.1 OBJETIVOS PEDAGÓGICOS

O presente projeto tem como desígnio consolidar os conhecimentos adquiridos ao longo do percurso académico, com especial enfoque na convergência interdisciplinar entre as unidades curriculares de Linguagens e Paradigmas da Programação (LPP) e Aplicações e Tecnologias Web (ATW). No âmbito de LPP, o objetivo pedagógico centra-se na aplicação prática de diferentes paradigmas de programação e na correta implementação de padrões de desenho de software (design patterns), visando a criação de um código estruturado, modular, escalável e de fácil manutenção. Paralelamente, no contexto de ATW, procura-se demonstrar a proficiência técnica na conceção e no desenvolvimento de aplicações Web dinâmicas e assíncronas, assentando numa arquitetura cliente-servidor robusta que assegure o processamento eficiente de dados e a otimização dos recursos computacionais.

1.2.2 OBJETIVOS OPERACIONAIS E FUNCIONAIS

Do ponto de vista operacional, o projeto propõe-se a desenvolver uma solução de software funcional e adaptada às necessidades da comunidade do ISPGAYA. Destaca-se a implementação do Clube de Leitura, materializado num catálogo virtual de livros com funcionalidades de pesquisa avançada e sistema de requisição direta via e-mail académico. Concomitantemente, estabelece-se a criação de um módulo para o Clube de Teatro dotado de uma bilheteira digital inovadora, que incorpora um mapa interativo de assentos, validação de disponibilidade em tempo real para prevenção de duplicações, e geração automática de bilhetes em formato PDF com código QR integrado. O sistema visa ainda disponibilizar galerias dinâmicas e históricos de atuações para a Tuna Académica, incorporar um motor de pesquisa e filtragem de programação cultural externa nas regiões de Gaia, Porto e Espinho e, por fim, integrar um assistente virtual baseado em Inteligência Artificial (Gemini) capaz de esclarecer dúvidas e interagir proativamente com os utilizadores da plataforma.

1.3 ESTRUTURA DO DOCUMENTO

O presente relatório encontra-se estruturado em dez capítulos que documentam de forma metódica o ciclo de vida do desenvolvimento da plataforma. Após esta Introdução, o Capítulo 2 procede ao levantamento e análise dos requisitos do sistema, seguindo-se o Capítulo 3, que aborda a usabilidade e o desenho da interface gráfica. O Capítulo 4 incide sobre a engenharia de dados e estruturação da base de dados, enquanto os Capítulos 5 e 6 expõem, respetivamente, os paradigmas de programação adotados e os padrões de desenho de software aplicados. O Capítulo 7 descreve as tecnologias escolhidas e o processo de implementação, sendo complementado pelo Capítulo 8, que elenca os mecanismos de segurança integrados na plataforma. O Capítulo 9 documenta as fases de testes realizadas para garantia de qualidade e, por último, o Capítulo 10 encerra o documento com as conclusões finais e possíveis perspectivas de evolução futura do projeto.

2. ANÁLISE E ESPECIFICAÇÃO DE REQUISITOS

2.1 DESCRIÇÃO GERAL DO SISTEMA

A arquitetura da plataforma "Laboratório Cultural" assenta num modelo cliente-servidor estruturado em dois ambientes distintos e complementares: o *Front-office* e o *Back-office*. Esta separação lógica e física garante uma delimitação clara entre a interface pública, destinada à fruição e interação por parte da comunidade, e a interface privada, desenhada para a gestão e administração da plataforma. O sistema é suportado por uma *stack* tecnológica robusta, utilizando Flask (Python) no *backend* para o processamento lógico e integração com a base de dados, e HTML5, Tailwind CSS e Vanilla JavaScript no *frontend*, proporcionando interfaces fluidas, responsivas e modularizadas.

No *Front-office*, o fluxo de navegação foi concebido para ser intuitivo e centrado na experiência do utilizador comum, que inclui estudantes, docentes, funcionários e eventuais visitantes externos. Neste ambiente, o utilizador dispõe de acesso imediato aos destaques culturais e pode navegar pelos diversos módulos temáticos. O percurso típico envolve a pesquisa no catálogo de livros do Clube de Leitura, a reserva de lugares na bilheteira digital do Clube de Teatro através de uma grelha interativa, e a consulta da agenda e galeria da Tuna Académica. A navegação é otimizada por mecanismos de pesquisa e filtragem assíncrona baseados em AJAX, permitindo que a listagem de eventos externos e notícias seja atualizada em tempo real sem necessidade de recarregar a página, culminando na possibilidade de interação direta com um chatbot de Inteligência Artificial para esclarecimento de dúvidas.

Por sua vez, o *Back-office* constitui o núcleo administrativo da plataforma, acessível exclusivamente a utilizadores com privilégios de Administrador mediante um sistema rigoroso de controlo de acessos e autenticação. O fluxo neste ambiente é focado na gestão operacional e na manutenção do conteúdo disponibilizado no *Front-office*. O gestor acede a um painel de controlo centralizado onde executa operações de criação, leitura, atualização e eliminação (CRUD) sobre todas as entidades do sistema, tais como obras literárias, peças de teatro, eventos, notícias e membros da Tuna. Adicionalmente, é neste espaço que o fluxo de aprovação se concretiza, permitindo ao Administrador rever, validar ou rejeitar as requisições de livros efetuadas pelos utilizadores, garantindo assim o normal funcionamento logístico do Laboratório Cultural.

2.2 LEVANTAMENTO DE REQUISITOS DO SISTEMA

2.2.1 REQUISITOS FUNCIONAIS (RF)

Os Requisitos Funcionais definem o comportamento esperado do sistema, detalhando as funcionalidades específicas, as interações com os utilizadores e o processamento de dados que a plataforma deve obrigatoriamente executar para cumprir o seu propósito. O processo de identificação destes requisitos baseou-se na análise das necessidades da comunidade académica do ISPGAYA e nos objetivos pedagógicos e operacionais propostos, traduzindo-se nas ações concretas que o *software* deve suportar. A Tabela 1 sistematiza as funcionalidades vitais para a operação do Laboratório Cultural.

Tabela 1- Requisitos Funcionais do Sistema

ID	Nome do Requisito	Descrição Detalhada
RF01	Apresentação Geral e Destaques	O sistema deve apresentar uma página inicial (Homepage) que agregue e destaque as atividades culturais mais relevantes, notícias recentes e os próximos eventos calendarizados.
RF02	Módulo de Biblioteca/Leitura	O sistema deve disponibilizar um catálogo virtual de livros que permita a consulta de obras e a sua requisição através da submissão do e-mail académico do utilizador.
RF03	Bilheteira Digital de Teatro	O sistema deve permitir a visualização de uma grelha interativa com o mapa físico de assentos, possibilitando a escolha de lugares específicos e a respetiva reserva sem ocorrência de duplicações.

RF04	Módulo da Tuna Académica	O sistema deve possuir uma secção dedicada à Tuna, apresentando uma listagem dos seus elementos, o histórico/agenda de atuações e uma galeria de fotos dinâmica.
RF05	Filtros de Eventos	O sistema deve integrar um mecanismo de pesquisa no <i>front-office</i> que permita filtrar eventos culturais externos por cidade (Gaia, Porto, Espinho), categoria e data.
RF06	Mapeamento e Filtro de Notícias	O sistema deve listar notícias e artigos relevantes, providenciando ferramentas de filtragem categorizada e cronológica para facilitar a pesquisa aos utilizadores.
RF07	Painel de Administração/Back-office	O sistema deve conter uma área administrativa restrita, acessível via <i>login</i> , que suporte operações CRUD completas para todas as entidades (livros, eventos, notícias, tuna, reservas e requisições).
RF08	Geração e Envio de Tickets PDF	Após a reserva de um bilhete de teatro, o sistema deve gerar automaticamente um bilhete em formato PDF, contendo um QR Code único, e enviá-lo para o e-mail do utilizador.

RF09	Chatbot de Inteligência Artificial	O sistema deve integrar um assistente virtual baseado no modelo Gemini IA, com capacidade de interagir em linguagem natural e responder a dúvidas sobre as atividades do portal.
-------------	------------------------------------	--

2.2.2 REQUISITOS NÃO FUNCIONAIS (RNF)

Os Requisitos Não Funcionais, ao invés de descreverem as ações do sistema, estabelecem as restrições e os critérios de qualidade subjacentes à sua arquitetura, segurança, desempenho e usabilidade. Estes requisitos são determinantes para assegurar que a plataforma seja robusta, escalável e ofereça uma experiência de utilização fiável e segura. A definição técnica destes parâmetros orienta as escolhas de infraestrutura e de padrões de desenvolvimento de *software*. A Tabela 2 apresenta as principais exigências técnicas do projeto.

Tabela 2- Requisitos Não Funcionais do Sistema

ID	Categoria	Descrição Técnica
RNF01	Segurança	O sistema deve aplicar o algoritmo de <i>hashing bcrypt</i> para o armazenamento de palavras-passe e implementar mecanismos de parametrização de <i>queries</i> e sanitização de <i>inputs</i> para prevenir ataques de <i>SQL Injection</i> (SQLi) e <i>Cross-Site Scripting</i> (XSS).

RNF02	Compatibilidade de Base de Dados	A camada de persistência de dados do sistema deve ser agnóstica e flexível, garantindo suporte operacional tanto para bases de dados SQLite (em ambiente de desenvolvimento) como MySQL (em ambiente de produção).
RNF03	Design Responsivo	A interface de utilizador (UI) deve adaptar-se dinamicamente às diferentes resoluções de ecrã, assegurando total compatibilidade e usabilidade em dispositivos móveis, <i>tablets</i> e computadores (<i>desktop</i>).
RNF04	Desempenho	O sistema deve utilizar chamadas assíncronas (AJAX) para a execução de buscas e aplicação de filtros no <i>front-office</i> , minimizando os tempos de carregamento e o tráfego de rede entre cliente e servidor.

2.3 ATORES DO SISTEMA E CASOS DE USO

Para a correta modelação das interações no sistema, identificam-se dois atores principais com papéis e níveis de permissão distintos. O **Utilizador Comum** (representando Estudantes, Docentes, Funcionários ou Visitantes) interage exclusivamente com a interface pública do portal, procurando consumir informação, reservar recursos culturais e esclarecer dúvidas. Em oposição, o **Administrador** (Gestor do Laboratório Cultural) é o ator responsável pela curadoria de conteúdos, moderação e aprovação de pedidos, possuindo credenciais de acesso ao *Back-office*.

A interação destes atores com o sistema traduz-se em fluxos lógicos denominados Casos de Uso (*Use Cases*). Destacam-se os seguintes processos fundamentais:

Requisitar Livro

Este caso de uso é iniciado pelo Utilizador Comum. Como pré-condição, a obra literária desejada deve estar registada no sistema com o estado de "disponível". O fluxo inicia-se com a seleção do livro no catálogo e o preenchimento de um formulário de requisição. O sistema valida os dados submetidos (verificando se o e-mail inserido possui um formato académico válido) e, caso a validação seja bem-sucedida, regista o pedido na base de dados com o estado "pendente". Por fim, o sistema notifica o Administrador no *Back-office* para posterior análise e aprovação, atualizando o estado do livro para evitar requisições concorrentes.

Reservar Bilhete de Teatro com Lugares

Iniciado pelo Utilizador Comum, este processo exige que o evento teatral esteja ativo e possua lugares vagos. O utilizador seleciona a peça e acede ao mapa de lugares (grelha interativa). Ao selecionar um ou mais assentos físicos e confirmar o pedido, o sistema efetua uma validação atómica na base de dados para garantir que os lugares não foram reservados em simultâneo por outro utilizador (controlo de concorrência e prevenção de duplicações). Sendo a verificação positiva, o sistema bloqueia os assentos, gera o bilhete eletrónico em formato PDF com a inclusão de um *QR Code* criptografado para validação à porta do evento, e aciona o serviço de envio (*SMTP*) para remeter o documento para o correio eletrónico do utilizador.

Gerir Eventos no Back-office

Ação exclusiva do Administrador. A pré-condição exige que o ator possua uma sessão autenticada ativa no painel administrativo. O fluxo contempla a navegação até ao módulo de gestão de programação cultural, onde o Administrador opta por criar, editar ou eliminar um evento. Ao inserir ou modificar dados (como título, data, cidade, categoria e imagem de cartaz), o sistema invoca mecanismos de validação (*backend* e *frontend*) para assegurar a integridade da informação e prevenir a injeção de código malicioso. Após a validação, a base de dados é atualizada e a transação é confirmada, refletindo-se imediatamente as alterações no *Front-office* sem necessidade de intervenção técnica adicional.

3. CONCEÇÃO DE INTERFACE E DESIGN (UI/UX)

3.1 IDENTIDADE INSTITUCIONAL E DESIGN SYSTEM

A conceção visual da plataforma "Laboratório Cultural" foi orientada pelo princípio da continuidade estética, procurando uma harmonização plena com a identidade institucional do ISPGAYA. Para garantir esta coerência, adotou-se um *Design System* que replica as diretrizes gráficas do portal corporativo da instituição, assegurando que o utilizador identifique de imediato a pertença da plataforma ao ecossistema académico do instituto. A paleta de cores é encabeçada pelo laranja institucional (#f37021), cor de forte carga energética e identitária, que foi estrategicamente aplicada em elementos de interação crítica, como botões de ação (*Call to Action* - CTAs), hiperligações, barras de destaque e indicadores de estados seleccionados.

Complementarmente, a estrutura cromática de suporte assenta num fundo escuro profundo (#1a1a1a) e numa escala de cinzas suaves para a delimitação de áreas de conteúdo. Esta escolha não é meramente estética; visa proporcionar uma interface sofisticada e moderna, reduzindo a fadiga visual e permitindo que as cores institucionais e os elementos multimédia ganhem maior relevância. O contraste é reforçado pelo uso de brancos puros em blocos de texto e divisores, garantindo o cumprimento das normas de acessibilidade e uma taxa de contraste que otimiza a legibilidade em diversas condições de luminosidade.

No que concerne à tipografia, procedeu-se à combinação de duas famílias de fontes do *Google Fonts* para estabelecer uma hierarquia visual clara e funcional. Para cabeçalhos e títulos de secções, utilizou-se a *Libre Baskerville*, uma fonte *Serif* clássica que transmite prestígio, autoridade e um carácter vincadamente literário e cultural, coadunando-se com a natureza do projeto. Para os elementos de interface, dados, tabelas, formulários e textos de apoio, foi seleccionada a fonte *Inter*. Trata-se de uma família *Sans-serif* de alta legibilidade, otimizada para ecrãs de computador e dispositivos móveis, que assegura uma leitura rápida e eficiente da informação técnica e operacional.

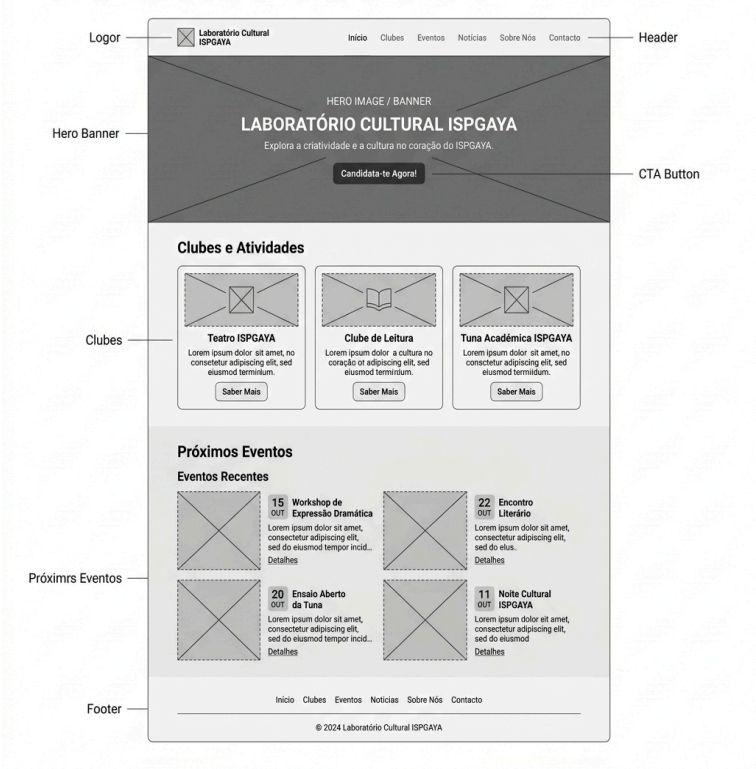
3.2 PLANEAMENTO VISUAL E PROTÓTIPOS (WIREFRAMES)

O processo de desenvolvimento da interface teve início com a criação de *wireframes* de baixa fidelidade. Esta etapa revelou-se fundamental para estruturar a hierarquia de informação e definir a disposição dos elementos funcionais sem a distração de componentes cromáticas ou texturas. Através destes protótipos, validou-se o fluxo de navegação e a organização espacial da plataforma, garantindo que as funcionalidades mais críticas estivessem sempre ao alcance do utilizador com o mínimo de cliques possível.

3.2.1 HOMEPAGE DO LABORATÓRIO CULTURAL

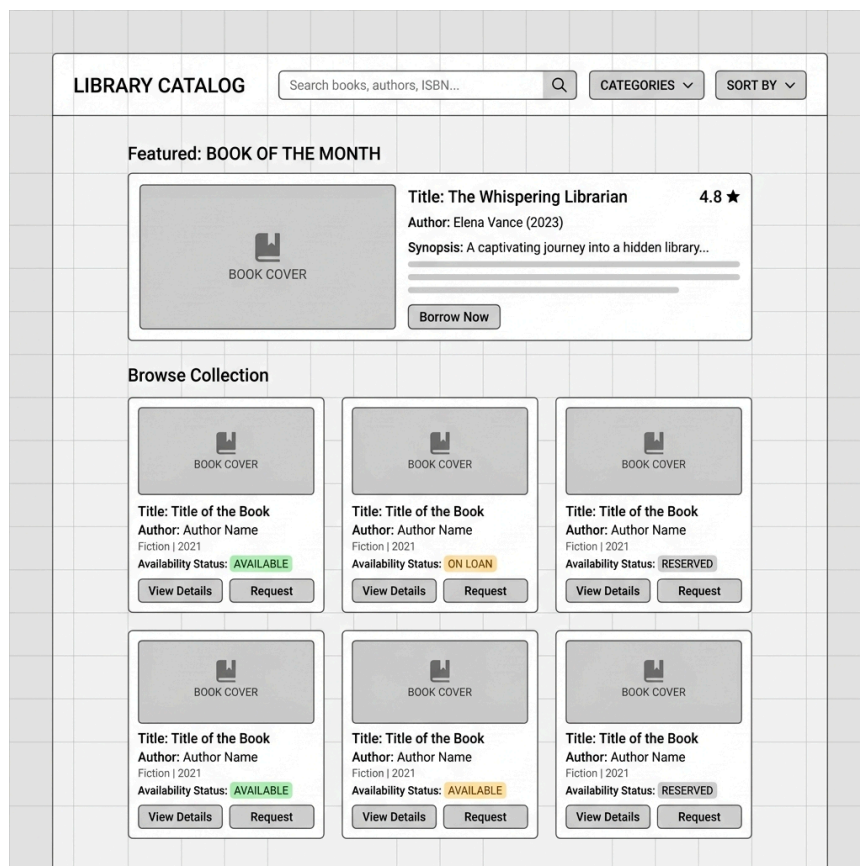
A página principal foi estruturada numa lógica de blocos modulares que guiam o utilizador através da oferta cultural. No topo, destaca-se uma Secção *Hero* com um *banner* dinâmico que apresenta a missão do Laboratório e imagens de grande formato em alta resolução. Imediatamente abaixo, uma grelha tripartida oferece acesso direto aos Clubes de Leitura, Teatro e Tuna. A página é complementada por uma listagem cronológica dos próximos eventos e notícias, culminando numa barra de consentimento de *cookies* que segue os padrões institucionais e as normas de privacidade vigentes.

HOMEPAGE WIREFRAME - LABORATÓRIO CULTURAL ISPGAYA



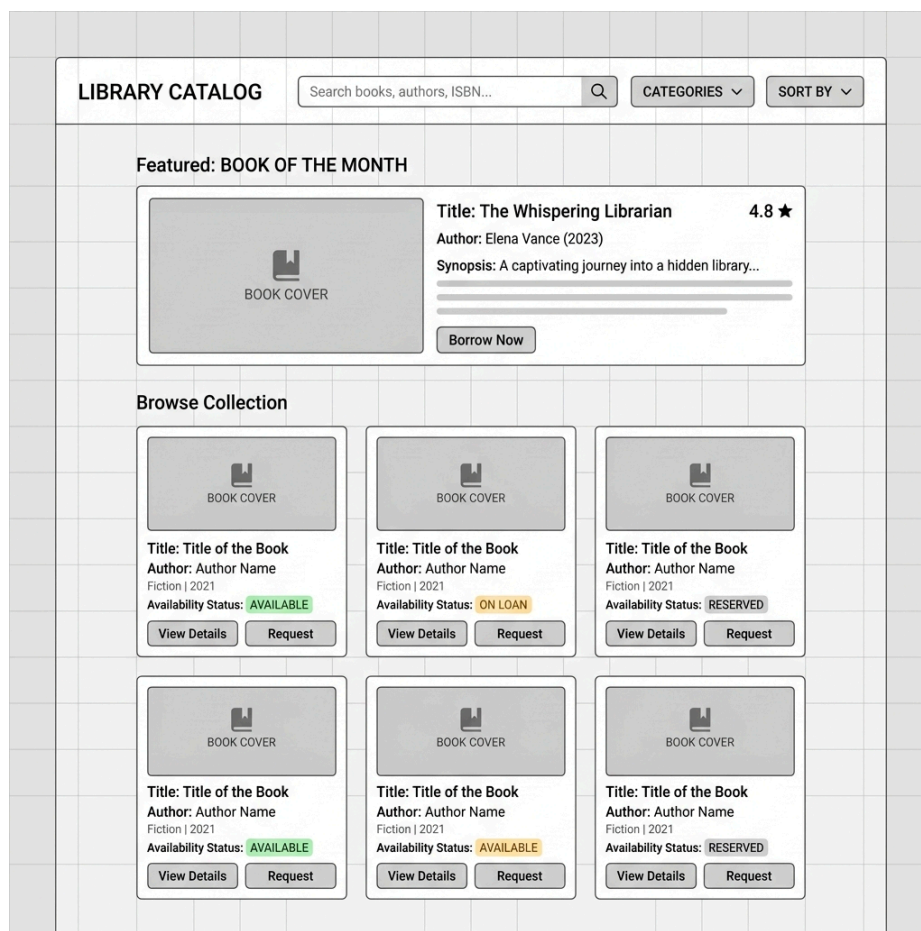
3.2.2 MÓDULO DE TEATRO: BILHETEIRA E MAPA DE LUGARES

O desenho do módulo de teatro focou-se na resolução de um problema logístico: a reserva de lugares físicos. A interface apresenta os detalhes do espetáculo e, centralmente, um mapa bidimensional representando a sala. Este mapa foi concebido como uma grelha de assentos operáveis, onde botões distintos indicam, através de códigos de cores, se o lugar está livre, ocupado ou selecionado pelo utilizador. Um painel lateral de resumo dinâmico atualiza, em tempo real, o número de lugares escolhidos e o custo total, precedendo o formulário de finalização que exige a validação do e-mail institucional.



3.2.3 BIBLIOTECA: CATÁLOGO VIRTUAL E EMPRÉSTIMOS

O catálogo do Clube de Leitura foi projetado para privilegiar a descoberta de obras. A interface inclui uma barra de pesquisa proeminente com filtros por categoria literária, um carrossel dinâmico de capas de livros e uma secção de destaque para o "Livro do Mês". A interação de requisição foi implementada através de um modal de sobreposição (*overlay*). Este componente surge sobre o catálogo quando o utilizador decide reservar uma obra, permitindo a inserção dos dados necessários sem que este perca o contexto da sua navegação principal no catálogo.



3.3 EXPERIÊNCIA DO UTILIZADOR (UX) E MICRO-INTERAÇÕES

A experiência do utilizador foi aprimorada através da implementação de micro-interações e técnicas de usabilidade avançadas no *front-end*. Para garantir uma navegação fluida em qualquer dispositivo, os menus de navegação utilizam transições responsivas que se adaptam automaticamente ao tamanho do ecrã. Foram aplicadas animações suaves via CSS, utilizando a propriedade *transition-all* com uma duração de 300 milissegundos, para suavizar a alteração de estados em botões e cartões de conteúdo. Na galeria da Tuna Académica, implementou-se um efeito visual onde o filtro de escala de cinzas é removido e a imagem sofre uma ligeira escala de ampliação ao passar o cursor (*hover*), conferindo dinamismo à navegação.

O sistema de *feedback* interativo foi igualmente priorizado para reduzir erros de submissão. Os formulários apresentam validação em tempo real, indicando erros de preenchimento através de bordas vermelhas e mensagens de aviso discretas, mas perceptíveis. Ecrãs de carregamento (*skeleton screens*) são utilizados durante as chamadas assíncronas para manter o utilizador informado sobre o processamento de dados. Por último, deu-se especial atenção à usabilidade móvel do mapa de assentos do teatro; em ecrãs de pequenas dimensões, a grelha de lugares mantém a sua integridade física através de um sistema de *scroll* horizontal dedicado, impedindo a quebra da estrutura visual e garantindo que o processo de reserva seja tão eficiente num telemóvel como num computador de secretária.

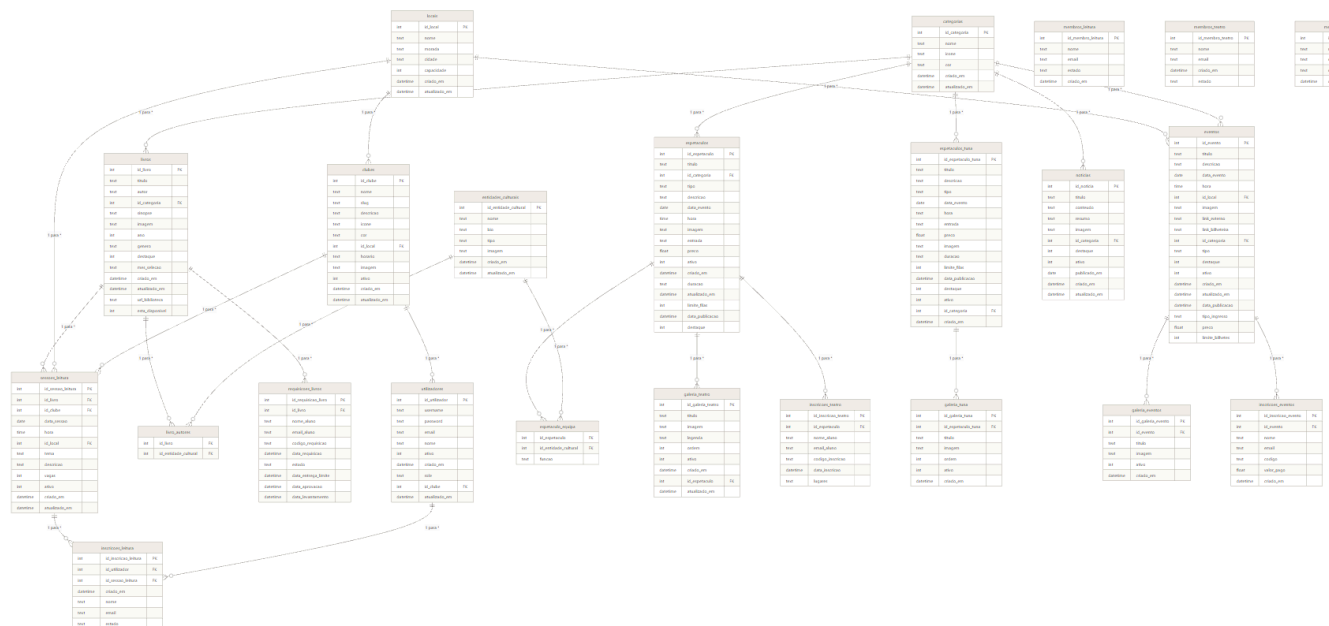
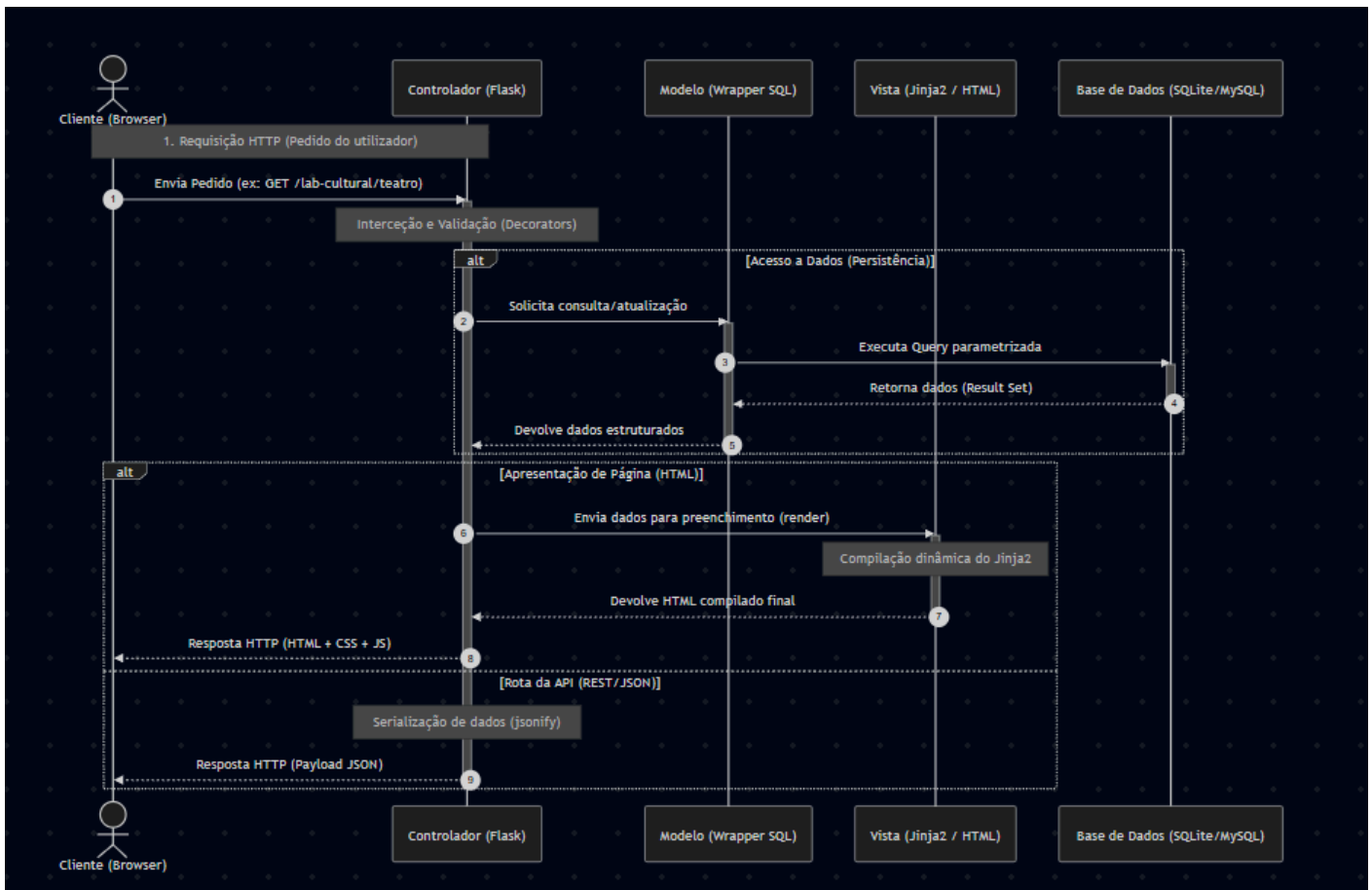
4. ENGENHARIA DE DADOS E ARQUITETURA DE SOFTWARE

4.1 ARQUITETURA DE SOFTWARE (MVC E BLUEPRINTS)

A arquitetura de software do "Laboratório Cultural" foi concebida segundo o padrão de desenho *Model-View-Controller* (MVC), implementado de forma idiomática sobre a *framework* Flask na linguagem Python. Esta abordagem arquitetónica permitiu a separação lógica das responsabilidades do sistema: a camada de Modelo (estruturas de dados e lógica de base de dados), a camada de Vista (*templates* HTML responsáveis pela interface com o utilizador) e a camada de Controlador (funções de roteamento e lógica de negócio que intermediam os pedidos do cliente e a base de dados). Esta separação garante um grau elevado de abstração, facilitando o isolamento de erros e promovendo a testabilidade do código-fonte.

Embora a aplicação adote uma estrutura monolítica, a complexidade do sistema exigiu um mecanismo rigoroso de organização de código para evitar a saturação do ponto de entrada principal do servidor. Para tal, recorreu-se à funcionalidade de *Blueprints* oferecida pelo Flask. Os *Blueprints* funcionam como módulos ou pacotes que agrupam rotas, *templates* e recursos estáticos sob um escopo comum. Esta estratégia de modularização revelou-se crucial para manter a legibilidade, escalabilidade e facilidade de manutenção do projeto, permitindo que diferentes componentes lógicos da aplicação fossem desenvolvidos, testados e acoplados de forma independente.

A divisão da aplicação materializou-se em quatro *Blueprints* fundamentais. O módulo frontoffice concentra toda a lógica de apresentação e roteamento das páginas públicas acedidas pelos utilizadores comuns. O back-office encapsula as operações de gestão, painel administrativo e os controladores de leitura e escrita (CRUD). O módulo *auth* centraliza a lógica de segurança, gerindo a autenticação de administradores, o controlo de sessões, encriptação de senhas e os respetivos encerramentos de sessão. Por fim, o *Blueprint* *api* foi estruturado para fornecer rotas *RESTful* exclusivas que devolvem dados no formato JSON, suportando as interações assíncronas (AJAX) no cliente, essenciais para pesquisas em tempo real e atualização dinâmica de interfaces sem recarregamento da página.



4.2 ESTRUTURA E ENGENHARIA DA BASE DE DADOS

4.2.1 JUSTIFICAÇÃO DAS DECISÕES TÉCNICAS DE ARMAZENAMENTO

Para a camada de persistência de dados, adotou-se uma estratégia híbrida focada na flexibilidade dos ambientes de execução. Durante a fase de desenvolvimento, optou-se pela utilização do motor SQLite. Esta decisão justifica-se pela sua natureza *serverless*, armazenando toda a base de dados num único ficheiro local (`lab_cultural.db`). Esta abordagem garante uma portabilidade autónoma e uma configuração célere, dispensando a instalação e configuração de pacotes de servidores locais pesados (como o XAMPP), o que agilizou significativamente a partilha de código e o teste entre diferentes máquinas.

No entanto, prevendo os requisitos do ambiente de produção, o sistema foi arquitetado para suportar o MySQL, um Sistema de Gestão de Bases de Dados Relacionais (SGBDR) vocacionado para elevada robustez, controlo rigoroso de acessos simultâneos e processamento eficiente de elevado volume de transações. Para viabilizar esta portabilidade bidirecional sem necessidade de reescrever consultas SQL, implementou-se uma camada de abstração no código (frequentemente materializada por um *Object-Relational Mapper* - ORM, ou adaptadores de dialetos). Esta camada permite que, através da simples alteração de uma variável de ambiente (a *Connection String*), o sistema comute automaticamente entre os dialetos SQL do SQLite e do MySQL, assegurando que as consultas e operações de base de dados permaneçam funcionais e otimizadas em ambos os motores.

4.2.2 DICIONÁRIO DE DADOS E TABELAS RELACIONAIS

A conceção da base de dados seguiu os princípios rigorosos da normalização, estruturando as 12 tabelas até à Terceira Forma Normal (3NF). Este processo visou eliminar redundâncias de armazenamento, mitigar anomalias de inserção, atualização e eliminação, e garantir que cada atributo não-chave dependesse única e exclusivamente da chave primária da respetiva tabela. A seguir, detalham-se os propósitos estruturais das entidades mais relevantes do modelo relacional:

- **UTILIZADORES:** Tabela destinada ao armazenamento das credenciais dos administradores. Implementa mecanismos de *Role-Based Access Control* (RBAC), definindo os diferentes níveis de privilégio (papéis) na gestão do *back-office*.
- **CATEGORIAS:** Entidade centralizadora que define a classificação temática transversal à plataforma, permitindo a categorização padronizada de eventos, livros e notícias num único local.
- **EVENTOS:** Regista a programação cultural externa e institucional. Armazena dados descritivos, temporais e espaciais (como cidade e local), fundamentais para a aplicação de filtros no portal.
- **NOTICIAS:** Tabela focada no armazenamento de artigos e novidades do ISPGAYA, contendo título, corpo de texto, data de publicação e identificadores para o autor e categoria.
- **LIVROS:** Catálogo bibliográfico do Clube de Leitura. Guarda a metainformação das obras (título, autor, ISBN) e regista de forma explícita o estado atual de requisição e disponibilidade.
- **INSCRICOES_TEATRO:** Regista as reservas efetuadas na bilheteira digital para os espetáculos do Clube de Teatro. Em plena conformidade com os princípios da Terceira Forma Normal (3NF), esta entidade armazena de forma atómica os dados identificativos do utilizador (através da chave estrangeira do e-mail académico) e o identificador do espetáculo. Em vez de gerir assentos físicos individualizados na sala, o sistema contabiliza unicamente a quantidade nominal de bilhetes reservados por transação. Esta abordagem simplifica as consultas relacionais de agregação (como a contagem e soma de bilhetes vendidos via funções **COUNT** e **SUM**), otimizando o cálculo da capacidade disponível em tempo real e eliminando a necessidade de serialização ou de junções (*JOINS*) complexas na base de dados.

4.2.3 INTEGRIDADE REFERENCIAL E CARDINALIDADES

A consistência e coesão dos dados são asseguradas pelo estabelecimento de Chaves Estrangeiras (*Foreign Keys*), que traduzem as lógicas de negócio em cardinalidades estritas no motor de base de dados. Observam-se relações do tipo "um-para-muitos" (1:N) como eixos estruturantes do modelo: uma determinada categoria pode estar associada a múltiplos livros, notícias e eventos; de igual modo, um espetáculo específico pode conter múltiplas inscrições de teatro. O modelo assegura também que tabelas dependentes possuam mapeamentos unívocos, garantindo que nenhum registo de requisição ou inscrição exista sem a respetiva obra ou espetáculo associado.

Para prevenir a existência de registos órfãos e salvaguardar a integridade referencial, foram configuradas regras de atuação perante eliminações no SGBDR. A aplicação de restrições de tipo **ON DELETE CASCADE** ou **ON DELETE RESTRICT** delega no motor relacional a responsabilidade de gerir anomalias. Por exemplo, a eliminação de um evento acarreta a eliminação em cascata das inscrições a ele associadas, garantindo que o volume de dados da plataforma se mantém coerente, sem acumulação de referências inválidas, mantendo assim a elevada fiabilidade da arquitetura de dados desenvolvida.

5. LINGUAGENS E PARADIGMAS DE PROGRAMAÇÃO (LPP)

5.1 PARADIGMA ORIENTADO A OBJETOS (POO)

A adoção do Paradigma Orientado a Objetos (POO) constituiu um pilar fundamental na arquitetura do "Laboratório Cultural", permitindo a modelação de entidades do mundo real em estruturas de dados lógicas e coesas. Através do recurso a classes, procedeu-se à abstração e representação das entidades centrais do sistema, tais como utilizadores, obras literárias e eventos culturais, encapsulando os seus atributos (estado) e métodos (comportamento). Esta abordagem promoveu um elevado grau de encapsulamento, ocultando a complexidade interna dos dados e garantindo que o seu estado apenas pudesse ser alterado através de interfaces bem definidas, o que aumenta substancialmente a segurança e a integridade da informação em memória.

No contexto da *framework* Flask, a POO manifestou-se igualmente através do uso extensivo dos mecanismos de herança e polimorfismo. A estruturação da aplicação tirou partido de classes base fornecidas pela *framework* e pelas suas extensões, permitindo a criação de componentes modulares que herdavam comportamentos padronizados (como rotas, vistas e gestão de sessões) e os adaptam às necessidades específicas do projeto. A herança facilitou a reutilização de código, evitando a duplicação de lógicas estruturais e promovendo uma manutenção centralizada, essencial para a escalabilidade da plataforma.

Para além da modelação de dados, o paradigma orientado a objetos foi aplicado na conceção de classes utilitárias responsáveis por abstrair a infraestrutura técnica. Um exemplo notório é a implementação de *wrappers* personalizados para a comunicação com a base de dados, como a classe `SQLiteCursorWrapper`. Esta estrutura encapsula a lógica de conexão, a gestão do cursor e a execução de interrogações SQL, assegurando que os controladores (*controllers*) da aplicação interajam com a persistência de dados de forma simplificada e segura, abstraindo-os da complexidade transacional e prevenindo o vazamento de recursos (*resource leaks*).

```
class SQLiteCursorWrapper:
    def __init__(self, db_connection):
        self.connection = db_connection
        self.cursor = None

    def __enter__(self):
        # Encapsula a inicialização do cursor
        self.cursor = self.connection.cursor()
        return self.cursor

    def __exit__(self, exc_type, exc_val, exc_tb):
        # Garante o fecho seguro dos recursos e gestão transacional
        if exc_type is None:
            self.connection.commit()
        else:
            self.connection.rollback()
        self.cursor.close()
```

5.2 PARADIGMA FUNCIONAL

Embora o Python seja predominantemente reconhecido pela sua natureza imperativa e orientada a objetos, a sua flexibilidade multiparadigma permitiu a integração de conceitos avançados do Paradigma Funcional na plataforma. Este paradigma foi adotado para maximizar a previsibilidade do código e minimizar a ocorrência de efeitos colaterais (*side-effects*), recorrendo ao uso de funções puras e à manipulação de estruturas de dados baseada na imutabilidade. Ao tratar as funções como "cidadãos de primeira classe" (*first-class citizens*), procedeu-se à passagem de funções como argumentos e ao seu retorno, o que simplificou o processamento de respostas de API (em formato JSON) e a gestão de dicionários complexos no servidor.

Uma das aplicações práticas mais evidentes deste paradigma consistiu no uso de expressões *lambda* combinadas com funções de ordem superior (*higher-order functions*), tais como `map()` e `filter()`. Estas funções anónimas de sintaxe reduzida foram empregues para a ordenação rápida e triagem de vetores de dados. Por exemplo, na extração e filtragem de eventos ou na ordenação cronológica de atuações da Tuna Académica, o recurso a expressões *lambda* permitiu aplicar transformações lógicas diretamente sobre as coleções, sem a necessidade de instanciar funções estáticas ou variáveis intermédias de controlo de estado.

Adicionalmente, destacou-se a utilização de *List Comprehensions*, uma funcionalidade idiomática do Python que incorpora a essência do paradigma funcional ao permitir a construção de novas coleções de forma declarativa e concisa. Esta técnica foi intensivamente aplicada na transformação e formatação de vetores de dados do *front-office*, como a desserialização e processamento da lista de assentos ocupados no módulo de teatro. Através desta abordagem funcional, gerou-se código altamente legível e performante, capaz de mapear e filtrar grandes volumes de dados de forma expressiva, respeitando a integridade das coleções de origem.

```
assentos_bd = ["Fila A-1", "Fila B-2", "Fila A-2", "Fila C-1"]

# Uso de List Comprehension para filtrar e formatar assentos imutavelmente
assentos_fila_a = [assento.upper() for assento in assentos_bd if assento.startswith("Fila A")]

# Uso de expressão lambda para ordenação de dicionários
eventos = [{"nome": "Peça de Teatro", "vagas": 5}, {"nome": "Concerto Tuna", "vagas": 120}]
eventos_ordenados = sorted(eventos, key=lambda e: e["vagas"], reverse=True)
```

5.3 PARADIGMA IMPERATIVO E PROCEDIMENTAL

Apesar da adoção de abstrações orientadas a objetos e funcionais, o Paradigma Imperativo manteve-se como a espinha dorsal para a gestão do fluxo de execução sequencial e das mutações de estado no servidor da aplicação Web. O raciocínio procedimental provou ser imprescindível para a concretização das lógicas de validação de formulários, controlo de acessos e operações de leitura e escrita em ficheiros, onde a instrução passo-a-passo ditada pelas estruturas de controlo e decisão (condicionais *if/elif/else*) modela rigorosamente a resposta da plataforma aos pedidos HTTP efetuados pelos clientes.

Neste enquadramento, as estruturas iterativas lineares (*for* e *while*) foram cruciais na execução de processos rotineiros e procedimentais. Um caso paradigmático reside no *script* de inicialização da aplicação (*start.py*), onde a conexão à base de dados, a configuração de variáveis de ambiente e a criação de tabelas ocorrem de forma estritamente sequencial. De igual modo, este paradigma foi o mais adequado para o preenchimento inicial de dados de simulação (*mock data*), onde ciclos imperativos garantem a inserção ordenada de registos de teste na base de dados, demonstrando

que o domínio sobre as alterações de estado continua a ser vital em arquiteturas de *software* de natureza transacional.

6. PADRÕES DE DESENHO DE SOFTWARE (DESIGN PATTERNS)

6.1 PADRÃO MODEL-VIEW-CONTROLLER (MVC)

A arquitetura do sistema "Laboratório Cultural" foi alicerçada no padrão de desenho arquitetónico *Model-View-Controller* (MVC), uma abordagem consagrada na engenharia de *software* que promove a separação rigorosa de conceitos (*separation of concerns*). No ecossistema de desenvolvimento suportado pela *framework* Flask e pela linguagem Python, a implementação deste padrão revelou-se instrumental para dissociar a lógica de negócio, a persistência de dados e a interface de utilizador. Esta segregação mitiga a complexidade intrínseca de aplicações de média e grande escala, permitindo que diferentes componentes sejam desenvolvidos, testados e mantidos de forma independente, o que eleva a escalabilidade e a resiliência do *software*.

Na componente de Modelo (*Model*), concentrou-se a responsabilidade pela gestão das conexões SQL e pela execução das regras de validação de dados antes da sua persistência na base de dados. O Controlador (*Controller*), por sua vez, atua como o orquestrador do sistema. Este componente foi materializado através da declaração de rotas encapsuladas em *Blueprints* temáticos (frontoffice, back-office, auth, api). Estes controladores intercetam os pedidos HTTP (*GET* e *POST*) originados pelo cliente, processam as regras de negócio, comunicam com a camada de modelo para obter ou modificar a informação necessária e, por fim, encaminham os dados processados para a camada de apresentação.

A componente de Vista (*View*) é estritamente responsável pela apresentação visual e interação com o utilizador, estando completamente isolada da lógica de processamento de dados. Esta camada foi implementada através de ficheiros de *templates* dinâmicos baseados em HTML5 e estilizados com o *framework* de CSS utilitário Tailwind CSS. A renderização final destas vistas é assegurada pelo motor de *templates* Jinja2, integrado nativamente no Flask, que recebe as variáveis injetadas pelos controladores e processa a geração do código HTML final que é entregue ao navegador do cliente. Este desacoplamento assegura que modificações na interface gráfica não exijam alterações no código lógico do servidor.

6.2 PADRÃO APPLICATION FACTORY

O padrão de desenho criacional *Application Factory* (Fábrica de Aplicações) foi adotado para gerir o ciclo de vida e a inicialização da instância da aplicação Flask. Em oposição à prática de instanciar a aplicação como uma variável global ao nível do módulo, este padrão recorre a uma função geradora, tipicamente designada por `create_app()` e localizada no ficheiro de inicialização do pacote principal (`app/__init__.py`). Esta abordagem encapsula toda a complexidade inerente à configuração do servidor, garantindo que o contexto da aplicação apenas é criado no momento em que é explicitamente invocado.

No interior desta função geradora, procede-se ao registo centralizado dos diversos *Blueprints*, à inicialização de extensões de terceiros e à configuração de variáveis de ambiente. A principal vantagem desta arquitetura reside na sua enorme flexibilidade: permite instanciar simultaneamente múltiplas versões da aplicação com parâmetros de configuração distintos. Torna-se, assim, possível alternar de forma transparente entre ambientes de desenvolvimento, produção e testes, carregando diferentes credenciais ou definições de base de dados sem necessidade de modificar o código estrutural.

Adicionalmente, a utilização da *Application Factory* é um facilitador crítico para a execução de testes de integração e testes unitários. Ao permitir a criação isolada de instâncias efémeras da aplicação (frequentemente acopladas a bases de dados em memória), os programadores garantem que os testes são executados num ambiente estéril, livre de interferências de estado global ou efeitos colaterais de execuções anteriores, cumprindo com rigor as boas práticas de engenharia de qualidade.

6.3 PADRÃO DECORATOR

Os decoradores (*Decorators*) em Python consubstanciam a aplicação de um padrão de desenho estrutural homónimo, projetado para estender e modificar o comportamento de funções ou métodos de forma dinâmica, sem necessidade de alterar o seu código-fonte original. Atuando como *wrappers* (envolventes) em torno de funções alvo, os decoradores permitem injetar lógica de processamento que é executada imediatamente antes ou depois da invocação da função principal, promovendo uma elevada reutilização de código e a aplicação do princípio DRY (*Don't Repeat Yourself*).

No desenvolvimento da plataforma, este padrão foi extensivamente utilizado quer através dos decoradores nativos do Flask, quer por intermédio de implementações personalizadas. O exemplo mais ubíquo é o decorador `@app.route()`, que interceta a definição de funções no controlador para as mapear e registar internamente na tabela de rotas do servidor HTTP. Para as necessidades de segurança, aplicaram-se decoradores como o `@login_required`, responsável por intercetar requisições HTTP e validar previamente a existência e validade de *cookies* de sessão, redirecionando automaticamente utilizadores não autenticados.

De forma a gerir as permissões avançadas no módulo administrativo, desenvolveu-se o decorador personalizado `@role_required`. Este componente implementa a verificação de controlo de acessos baseado em papéis (RBAC). Antes de o controlador do *back-office* executar qualquer operação sensível (como eliminar utilizadores ou validar requisições), este decorador inspeciona os privilégios associados à sessão atual. Esta abordagem centraliza a lógica de autorização, mantendo os controladores limpos, focados exclusivamente na sua lógica de negócio, e garantindo que as regras de segurança são aplicadas de forma elegante e infalível.

6.4 PADRÃO REPOSITORY E ABSTRAÇÃO DE ACESSO A DADOS

O padrão *Repository* atua como um intermediário entre a lógica de domínio da aplicação e a camada de mapeamento e acesso a dados. O seu principal desígnio é isolar a complexidade das consultas SQL (*Structured Query Language*) e ocultar os detalhes físicos da infraestrutura da base de dados subjacente. No projeto do Laboratório Cultural, este padrão materializou-se no desenvolvimento de *wrappers* de dados personalizados, como a classe `SQLiteCursorWrapper`, que centralizam e padronizam a forma como os controladores comunicam com o motor de armazenamento.

Esta abstração revelou-se particularmente estratégica para suportar a arquitetura híbrida de base de dados do projeto. O *wrapper* é capaz de intercetar as chamadas SQL e, de forma transparente para o desenvolvedor, traduzir os parâmetros de dialeto em conformidade com o motor em utilização — por exemplo, comutando entre o marcador posicional `%s` exigido pelo MySQL em ambiente de produção e o caractere `?` utilizado pelo SQLite em ambiente de desenvolvimento local. Desta forma, assegura-se que a aplicação se mantém agnóstica em relação ao sistema de gestão

de base de dados, facilitando migrações futuras e garantindo a resiliência do acesso aos dados.

7. TECNOLOGIAS E IMPLEMENTAÇÃO WEB (ATW)

7.1 IMPLEMENTAÇÃO DO FRONT-END

7.1.1 ESTRUTURAÇÃO HTML5 E ESTILIZAÇÃO TAILWIND CSS

A estruturação da interface pública e administrativa da plataforma foi integralmente desenvolvida com recurso a HTML5, privilegiando a utilização de marcação semântica (*tags* como `<header>`, `<nav>`, `<section>`, `<article>` e `<footer>`). Esta abordagem garante não só a acessibilidade do portal para leitores de ecrã e tecnologias de apoio, mas também otimiza a indexação do conteúdo pelos motores de busca (SEO). A hierarquia do Document Object Model (DOM) foi desenhada para ser limpa e modular, suportando de forma eficiente a injeção de dados dinâmicos provenientes do servidor.

Para a componente de estilização, optou-se pela *framework* Tailwind CSS, que promove o paradigma *utility-first*. Em oposição a bibliotecas tradicionais baseadas em componentes pré-fabricados, como o Bootstrap, esta escolha tecnológica justificou-se pela necessidade de uma maior liberdade geométrica e visual, evitando a padronização estética limitativa e permitindo uma replicação mais fiel do *design system* do ISPGAYA. Adicionalmente, a arquitetura do Tailwind confere uma vantagem de *performance* crítica: permite uma otimização rigorosa do peso do ficheiro final em ambiente de produção através de mecanismos de purga (*PurgeCSS*), que eliminam automaticamente classes não utilizadas. Através de uma configuração customizada, foi possível mapear de imediato a paleta de cores corporativas (como o laranja institucional) e construir interfaces nativamente responsivas com prefixos de *media queries* no código HTML, garantindo a adaptação fluida a dispositivos móveis sem a necessidade de manter extensos e complexos ficheiros CSS independentes.

7.1.2 INTERATIVIDADE ASSÍNCRONA COM VANILLA JAVASCRIPT (FETCH API)

A dinamização da interface de utilizador no lado do cliente dispensou a utilização de *frameworks* de JavaScript pesadas (como React ou Angular), optando-se pelo uso de *Vanilla JavaScript* moderno (ES6+). Esta decisão técnica visou reduzir a sobrecarga da aplicação, mantendo o *front-end* ágil e performante. A interatividade principal assenta na manipulação direta do DOM e na comunicação assíncrona com o servidor através da *Fetch API*, permitindo a atualização cirúrgica de componentes de interface sem que ocorra o recarregamento total da página web (*refresh*).

Um dos exemplos mais notórios desta implementação verifica-se nos módulos de pesquisa e filtragem de livros e de eventos culturais. À medida que o utilizador digita um termo de pesquisa ou altera os filtros de categoria e cidade, o JavaScript capta esses eventos e dispara pedidos assíncronos (AJAX) para os *endpoints* da API interna. A resposta, devolvida no formato JSON, é imediatamente processada para repovoar as grelhas de resultados. Este mecanismo inclui também a lógica de *autocomplete* (preenchimento automático), oferecendo sugestões em tempo real que enriquecem substancialmente a experiência de navegação.

Adicionalmente, o *Vanilla JavaScript* foi fundamental para o controlo dinâmico do mapa de assentos da bilheteira de teatro. Através da monitorização de cliques na grelha bidimensional, o *script* altera os estados visuais dos botões (livre, selecionado, indisponível), gere matrizes de dados em memória com os lugares escolhidos e atualiza contadores de bilhetes e preços totais de forma reativa. No momento da finalização, este estado é empacotado num objeto estruturado e submetido via *Fetch API* para o processamento transacional no *backend*.

7.2 IMPLEMENTAÇÃO DO BACKEND

7.2.1 ROTAS FLASK E MOTOR DE TEMPLATES JINJA2

O servidor da aplicação foi implementado em Python utilizando a *micro-framework* Flask, cuja arquitetura minimalista foi complementada através do registo de *Blueprints* modulares. O mapeamento de URLs é gerido de forma declarativa através de decoradores (como `@app.route`), permitindo a criação de rotas estáticas e dinâmicas. O uso de rotas dinâmicas, que capturam parâmetros diretamente do URL (por exemplo, o identificador único de um evento para renderizar a sua página de detalhes), facilitou o desenvolvimento de uma arquitetura baseada nos princípios REST, separando logicamente as requisições de leitura (GET) das submissões de formulários (POST).

Para a renderização das páginas devolvidas ao cliente, recorreu-se ao Jinja2, o motor de *templates* integrado no Flask. A principal vantagem técnica desta ferramenta reside no seu sistema de herança. Criou-se um *template* base (`base.html`) que contém as fundações estruturais do portal — como cabeçalhos, menus de navegação, estilos e rodapés — do qual todos os restantes *templates* herdam. Através da definição de blocos dinâmicos (`{% block content %}`), apenas o conteúdo específico de cada página é processado e injetado, reduzindo drasticamente a duplicação de código e simplificando a manutenção e atualização da interface global do projeto.

7.2.2 INTEGRAÇÃO DE DADOS E OPERAÇÕES CRUD NO BACK-OFFICE

O núcleo operacional da plataforma reside no seu painel administrativo (*Back-office*), onde se implementou um sistema robusto de operações CRUD (Criar, Ler, Atualizar, Eliminar). Este painel, protegido por mecanismos de autenticação e verificação de papéis (RBAC), fornece interfaces controladas para que o administrador possa gerir o ciclo de vida de todas as entidades do modelo de dados. As interações desencadeiam funções Python no *backend* que executam instruções SQL validadas, garantindo a integridade dos dados antes da sua persistência na base de dados relacional.

A implementação abrange a gestão completa do catálogo de livros, a calendarização de eventos externos, a publicação de notícias, a atualização da galeria e dos

elementos da Tuna Académica, e a configuração de espetáculos de teatro. Para cada uma destas entidades, foram desenvolvidos formulários de submissão e edição que integram validações estritas no servidor, prevenindo a introdução de dados mal formatados ou vulnerabilidades de segurança. A eliminação de registos foi desenhada de modo a respeitar a integridade referencial, impedindo a remoção de entidades que possuam dependências críticas ativas.

A arquitetura monolítica e a centralização da base de dados garantem que qualquer alteração efetuada através das operações CRUD no painel de administração produza efeitos instantâneos. Uma vez que o *front-office* e o *back-office* partilham a mesma fonte de verdade (*Single Source of Truth*), a adição de uma nova peça de teatro ou a aprovação de uma requisição de leitura reflete-se de imediato na interface pública. Esta sincronização contínua elimina a necessidade de atualizações manuais no código fonte e assegura a atualidade da informação disponibilizada à comunidade académica.

7.3 FUNCIONALIDADES OPCIONAIS E INOVADORAS (DIFERENCIADORES)

7.3.1 BILHETEIRA DE TEATRO INTERATIVA E CONTROLO DE CONCORRÊNCIA

A bilheteira digital foi desenhada para gerir de forma ágil a lotação dos espetáculos. A inovação técnica reside no algoritmo de controlo de concorrência desenvolvido no *backend* Flask para prevenir situações de *overbooking* (sobrelocação). Uma vez que o modelo de dados regista a quantidade nominal de bilhetes requisitados e não assentos físicos individualizados, o sistema necessita de assegurar de forma rigorosa que o somatório de reservas não ultrapassa a capacidade total estipulada para o espetáculo. Para mitigar anomalias transacionais, como *race conditions* (condições de corrida), a plataforma inicia uma transação atómica durante o processamento do pedido HTTP **POST** de reserva, recorrendo a um mecanismo de bloqueio pessimista (*Pessimistic Locking*). Através da instrução SQL **SELECT ... FOR UPDATE**, o sistema bloqueia temporariamente a linha referente ao espetáculo específico na tabela principal. Este bloqueio garante que, caso múltiplos utilizadores submetam pedidos de reserva no exato mesmo milissegundo, o motor da base de dados obriga as transações subsequentes a aguardar o desfecho da primeira. Após garantir o bloqueio exclusivo, o sistema recalcula em tempo real a lotação, verificando se a capacidade restante comporta a nova requisição. Se a validação for positiva, a transação é confirmada e o

número de bilhetes vendidos é atualizado; caso a lotação seja excedida, a transação é revertida (*rollback*) e o utilizador notificado, garantindo a integridade dos dados diretamente ao nível do escalonador do Sistema de Gestão de Base de Dados (SGBD).

7.3.2 GERAÇÃO AUTOMÁTICA DE TICKETS PDF E INCORPORAÇÃO DE QR CODES

Após a confirmação de uma reserva de teatro, o sistema desencadeia um *pipeline* automatizado para a geração do bilhete digital. Utilizando a biblioteca Python `qrcode`, o servidor cria uma matriz bidimensional contendo uma assinatura criptográfica única que representa a inscrição. Simultaneamente, a biblioteca `fpdf2` instancia um documento PDF em branco, desenhando dinamicamente o *layout* do bilhete com os dados do evento, o nome do utilizador, a identificação dos lugares e, por fim, a Incorporação do código QR gerado. De forma a otimizar os recursos do servidor, o ficheiro resultante é mantido num fluxo de *bytes* em memória RAM, o que permite anexá-lo e enviá-lo de imediato por e-mail sem necessitar de armazenamento físico em disco.

7.3.3 INTEGRAÇÃO DA API GOOGLE GEMINI PARA O ASSISTENTE VIRTUAL

Com o intuito de apoiar os alunos nas suas interações, o Laboratório Cultural integra um assistente virtual potenciado pelo modelo de Inteligência Artificial Google Gemini. Por motivos de segurança, a comunicação não é feita diretamente do navegador para os servidores da Google, o que exporia a chave da API. O *front-end* envia a dúvida do utilizador via *Fetch API* para um *endpoint* interno do Flask. Este *endpoint* atua como um *proxy* seguro: injeta a chave privada de API armazenada nas variáveis de ambiente e anexa instruções prévias de contexto (*system prompts*) — delimitando o escopo das respostas exclusivamente às regras, clubes e eventos do ISPGAYA — devolvendo seguidamente a resposta estruturada ao cliente.

7.3.4 COMUNICAÇÃO AUTOMÁTICA POR E-MAIL (SERVIÇO SMTP)

A conclusão dos processos interativos do portal assenta numa comunicação eficaz com os utilizadores, concretizada através da configuração de um cliente de envio de correio eletrónico baseado no protocolo SMTP (*Simple Mail Transfer Protocol*). No ambiente Flask, esta ligação foi parametrizada com credenciais seguras, definindo o servidor de correio, o respetivo porto e os protocolos de encriptação (SSL/TLS), de modo a permitir ao sistema enviar mensagens de forma autónoma através do e-mail institucional do projeto.

Esta funcionalidade é acionada assincronamente em dois momentos críticos: na expedição imediata dos bilhetes em formato PDF após a reserva num espetáculo e no envio de códigos de levantamento quando o administrador aprova uma requisição de um livro no Clube de Leitura. A automação deste fluxo transacional encerra o ciclo de utilização de forma fluida, providenciando ao estudante um comprovativo documental tangível das suas ações na plataforma e elevando o padrão de serviço logístico do Laboratório Cultural.

8. SEGURANÇA

8.1 PREVENÇÃO DE CROSS-SITE SCRIPTING (XSS)

A vulnerabilidade de *Cross-Site Scripting* (XSS) ocorre quando uma aplicação web permite a injeção de *scripts* maliciosos em páginas visualizadas por outros utilizadores. Esta falha de segurança possibilita que um atacante execute código arbitrário (geralmente JavaScript) no navegador da vítima, comprometendo sessões, roubando credenciais ou alterando o comportamento e a aparência do portal. Devido à natureza interativa do "Laboratório Cultural", que permite a submissão de dados através de formulários (como a criação de eventos ou notícias), a mitigação deste vetor de ataque assumiu carácter prioritário.

Para combater este risco, implementou-se uma estratégia de defesa em profundidade baseada em duas camadas de proteção. No *backend*, desenvolveu-se o módulo `app/utils/security.py`, onde se destaca a função `sanitize()`. Esta rotina recorre à biblioteca nativa `html.escape()` do Python para converter sistematicamente caracteres sensíveis e de controlo HTML (tais como `<`, `>`, `&`, `"` e `'`) nas suas respetivas entidades seguras. Adicionalmente, concebeu-se a função `sanitizar_dict()`, que itera sobre

estruturas de dados complexas, garantindo a limpeza rigorosa de múltiplos campos de um formulário em simultâneo antes do seu processamento.

A segunda camada atua diretamente na componente de *View (front-end)*. O motor de *templates* Jinja2, integrado nativamente na *framework* Flask, foi configurado para aplicar o mecanismo de *auto-escape* por omissão. Isto significa que qualquer variável injetada e renderizada nos ficheiros HTML é automaticamente tratada como texto estático. Esta redundância de segurança assegura que, mesmo perante um lapso na sanitização do lado do servidor, o navegador do cliente jamais interpretará *inputs* de utilizadores como instruções de código executável.

```
import html

def sanitize(text):
    """Converte caracteres de risco em entidades HTML seguras."""
    if not text:
        return text
    return html.escape(str(text))

def sanitizar_dict(data_dict):
    """Aplica a sanitização a todos os valores de um dicionário."""
    return {key: sanitize(value) for key, value in data_dict.items()}
```

8.2 PREVENÇÃO DE SQL INJECTION

A injeção de código SQL (*SQL Injection* ou *SQLi*) figura entre as ameaças mais críticas em sistemas de bases de dados relacionais. Esta vulnerabilidade explora a ausência de segregação entre os dados fornecidos pelo utilizador e a estrutura da consulta SQL, permitindo que *inputs* manipulados alterem a semântica da *query*. Tal cenário pode resultar em acessos não autorizados, exfiltração de dados sensíveis ou na corrupção total e eliminação dos registos da base de dados.

A erradicação desta vulnerabilidade no projeto processou-se através do banimento absoluto da concatenação direta de *strings* na construção de interrogações SQL. Em substituição, impôs-se o uso exclusivo de *queries* parametrizadas. Através da utilização de marcadores de posição (*placeholders*), como *?* e *%s*, o motor da base de dados é instruído a compilar a estrutura lógica da interrogação de forma isolada,

tratando os valores fornecidos estritamente como literais (dados) e nunca como comandos executáveis.

Para garantir a coerência deste paradigma face à arquitetura híbrida de bases de dados do sistema, a parametrização é coordenada pela classe utilitária `SQLiteCursorWrapper`. Este componente de *software* atua como um intercetor em tempo de execução (*runtime*), identificando a presença de sintaxe orientada ao MySQL (uso de `%s`) e procedendo à sua tradução dinâmica para a sintaxe do SQLite (uso de `?`) quando em ambiente de desenvolvimento, assegurando uma parametrização estável e intransponível independentemente do dialeto utilizado.

```
# Exemplo de consulta parametrizada que previne SQL Injection
def obter_utilizador_por_email(cursor, email_fornecido):
    query = "SELECT * FROM utilizadores WHERE email = %s"
    # O motor de BD trata 'email_fornecido' estritamente como texto,
    # neutralizando qualquer tentativa de injetar comandos SQL.
    cursor.execute(query, (email_fornecido,))
    return cursor.fetchone()
```

8.3 CRIPTOGRAFIA E ARMAZENAMENTO SEGURO DE PASSWORDS

O armazenamento de credenciais de autenticação em texto limpo ou o recurso a algoritmos de *hashing* obsoletos (como MD5 ou SHA1) constitui uma negligência de segurança grave, dado que a sua vulnerabilidade a colisões e a ataques de força bruta, suportados por poder computacional moderno, está largamente documentada. Para garantir a integridade e a confidencialidade das credenciais dos administradores, optou-se pela utilização da biblioteca criptográfica `bcrypt`, um algoritmo derivado do *Blowfish*, concebido especificamente para ser computacionalmente exigente e resistente à aceleração em *hardware*.

O processo de cifra inicial recorre à função `bcrypt.hashpw`. Uma das propriedades fundamentais deste algoritmo é a geração Automática de um *salt* único e aleatório para cada palavra-passe, que é posteriormente incorporado no *hash* resultante. O *salting* impossibilita a viabilidade de ataques baseados em tabelas pré-computadas (*Rainbow Tables*), garantindo que palavras-passe idênticas resultem sempre em sumários criptográficos distintos na base de dados.

Durante a autenticação, a verificação processa-se através da função `bcrypt.checkpw`. Esta rotina executa uma comparação criptográfica sofisticada que atua em tempo constante (*constant-time comparison*). Esta característica técnica é vital para prevenir ataques de *timing* (*Timing Attacks*), nos quais intervenientes maliciosos analisam variações microscópicas no tempo de resposta do servidor para deduzir iterativamente o comprimento e o conteúdo de chaves ou *hashes* armazenados.

```
import bcrypt

def criar_hash_password(password_texto_limpo):
    """Gera um salt único e encripta a password com bcrypt."""
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password_texto_limpo.encode('utf-8'), salt)

def verificar_credenciais(password_input, hash_guardado):
    """Compara a password submetida com o hash de forma segura."""
    return bcrypt.checkpw(password_input.encode('utf-8'), hash_guardado)
```

8.4 VALIDAÇÃO DE DADOS E RESTRIÇÃO DE UPLOADS

A fiabilidade e estabilidade de um sistema informático estão intrinsecamente dependentes da qualidade dos dados que processa. A validação de *inputs* do lado do cliente (via HTML e JavaScript) serve o propósito de melhorar a experiência do utilizador, porém, revela-se insuficiente como medida de segurança primária, visto poder ser facilmente contornada. Por conseguinte, implementou-se um regime rigoroso de validação e sanitização do lado do servidor (em Python), assegurando a integridade e a conformidade da informação perante a lógica de negócio estipulada.

O tratamento da informação submetida inclui o recurso a Expressões Regulares (*Regex*) para a verificação de formatos textuais, ilustrado pela função `validar_email()`, que restringe a requisição de livros exclusivamente ao domínio académico (`@ispgaya.pt`). Paralelamente, implementaram-se as funções `validar_data()` e `validar_hora()`, fundamentadas no módulo nativo `datetime`, garantindo o cumprimento de matrizes cronológicas, como o formato YYYY-MM-DD, evitando erros de conversão no SGBD durante o registo de eventos ou espetáculos.

Ao nível da gestão de ficheiros multimédia associados às atividades culturais, introduziu-se um estrito controlo através da função auxiliar `allowed_file()`. Esta rotina

procede à verificação exaustiva da extensão do ficheiro remetido, aceitando apenas formatos de imagem padrão (como png, jpg, jpeg, gif e webp). De modo a mitigar ataques de Negação de Serviço (DoS) consubstanciados na exaustão de largura de banda e capacidade de armazenamento, configurou-se a diretiva global `MAX_CONTENT_LENGTH` do Flask para 5 MB, o que determina a rejeição prematura e automática de pacotes que ultrapassem a dita dimensão.

8.5 MECANISMOS DE AVALIAÇÃO DE ROBUSTEZ DE PASSWORDS

A complexidade e a força das palavras-passe constituem o eixo central na defesa de interfaces restritas, atenuando a probabilidade de comprometimento derivado de ataques por dicionário ou de dedução social. Nesse sentido, desenvolveu-se o *endpoint* `/api/v1/avaliar-password`, dotado da capacidade de efetuar a análise de robustez destas credenciais em tempo real, sem que as mesmas venham a ser temporariamente guardadas na sessão do lado do servidor.

Os critérios de avaliação operam com base na análise quantitativa e qualitativa, exigindo cumulativamente dimensões mínimas de comprimento e um modelo de heterogeneidade de caracteres que compreenda a intercalação de minúsculas, maiúsculas, numerais e símbolos especiais. Adicionalmente, implementou-se um modelo proativo de rejeição que cruza a palavra-passe candidata com uma lista negra (*blacklist*) de credenciais manifestamente vulneráveis e frequentes no contexto global, bloqueando desde o primeiro momento as combinações consideradas "fracas".

8.6 ENGENHARIA DE SESSÕES E ENCERRAMENTO SEGURO (LOGOUT)

A transição entre o estado anónimo e as áreas de administração exigiu uma modelação acurada no processo de engenharia de sessões da plataforma. A arquitetura suportada pelo Flask promove o recurso a *cookies* de sessão assinados com primitivas criptográficas. A solidez deste mecanismo emana da chave de ambiente estática e secreta, alocada na variável global `SECRET_KEY`, cujo propósito reside em detetar inequivocamente quaisquer tentativas de adulteração de *payload* por manipulações de requisições no *client-side*. Os componentes lógicos designados como decoradores de segurança interceptam cada pedido de modo a analisar a integridade do *cookie* e o correspondente perfil RBAC previamente codificado.

O procedimento de *logout* transcende a simples expiração visual da ligação do cliente. Invocando o controlador respetivo, o sistema processa a diretiva `session.clear()`. Esta intervenção dita a destruição irreversível dos dados na cache e a invalidação

criptográfica do identificador transacional daquele utilizador autenticado. Deste modo, impossibilita-se que uma cópia retida do *cookie* possa vir a ser reaproveitada (ataques de *Session Replay*), consolidando as boas práticas sobre a minimização do tempo de vida e da eliminação de vestígios confidenciais após o fecho da sessão de trabalho.

8.7 MAPEAMENTO DE RISCOS (MATRIZ OWASP TOP 10)

Para assegurar o rigor científico e o alinhamento metodológico com os padrões globais da indústria da cibersegurança, o desenho da arquitetura e as estratégias defensivas da plataforma "Laboratório Cultural" foram referenciadas face ao projeto OWASP (*Open Worldwide Application Security Project*). A matriz do OWASP Top 10 identifica as vulnerabilidades web mais críticas e generalizadas, fornecendo um enquadramento essencial para o desenvolvimento seguro de *software*. A Tabela 3 sumariza o mapeamento das categorias de risco identificadas e as respetivas metodologias de mitigação instituídas.

Tabela 3- Matriz de Mapeamento OWASP Top 10 e Estratégias de Mitigação

Categoria OWASP	Descrição do Risco	Estratégia de Mitigação no Laboratório Cultural
A01:2021-Broken Access Control	Falhas na verificação de permissões que permitem a utilizadores acederem a recursos não autorizados.	Aplicação de Controlo de Acessos Baseado em Papéis (RBAC) e uso do decorador personalizado <code>@role_required</code> nas rotas do painel administrativo.
A02:2021-Cryptographic Failures	Exposição de dados sensíveis por armazenamento em texto limpo ou uso de criptografia fraca.	Hasheamento de todas as passwords através do algoritmo <code>bcrypt</code> com geração de <code>salts</code> e assinatura criptográfica dos <code>cookies</code> de sessão com <code>SECRET_KEY</code> .

A03:2021-Injection	Injeção de dados hostis para induzir a execução de comandos maliciosos no servidor ou base de dados.	Uso exclusivo de <i>queries</i> parametrizadas (<code>?</code> , <code>%s</code>), classe <code>SQLiteCursorWrapper</code> e sanitização de <i>inputs</i> (XSS) com as funções <code>sanitize()</code> e <i>auto-escape</i> .
A04:2021-Insecure Design	Falhas arquiteturais estruturais que abrem precedentes para ataques ao nível lógico ou financeiro.	Restrição de processamento de ficheiros (limite de tamanho de 5 MB por submissão com <code>MAX_CONTENT_LENGTH</code>) e lógica transacional no teatro para impedir dupla reserva.
A07:2021-Identification and Authentication Failures	Comprometimento de credenciais devido à fraca proteção, permitindo assunção fraudulenta de identidades.	Mecanismo de invalidação absoluta de sessões (<code>session.clear()</code>) e <i>endpoint</i> de validação que exige passwords robustas e bloqueia <i>inputs</i> pertencentes a <i>blacklists</i> .

8.8 SEGURANÇA NA INTEGRAÇÃO DA API GOOGLE GEMINI

A integração do assistente virtual baseado em Inteligência Artificial introduziu vetores de ataque específicos, exigindo uma arquitetura de segurança dedicada em três frentes: proteção de credenciais, mitigação de manipulação de contexto e prevenção de exaustão de recursos.

O primeiro pilar incide sobre a gestão de segredos. A chave de acesso privada (API Key) da Google nunca é declarada em texto limpo no código-fonte (*hardcoded*). Em conformidade com as boas práticas de desenvolvimento seguro, a credencial é injetada no sistema exclusivamente através de Variáveis de Ambiente (*Environment Variables*). Em ambiente de desenvolvimento, esta configuração é mantida num ficheiro `.env` alojado na raiz do projeto, o qual é estritamente excluído do sistema de controlo de versões (Git) através da diretiva `.gitignore`. Em tempo de execução, o *backend* Flask recorre ao módulo nativo `os` (ex: `os.getenv()`) para carregar a chave em memória, garantindo que qualquer eventual fuga do código-fonte não resulta no comprometimento financeiro ou roubo da identidade da aplicação.

O segundo vetor crítico diz respeito aos Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs), nomeadamente a Injeção de Instruções (*Prompt Injection*). Esta técnica ocorre quando um utilizador submete comandos projetados para reescrever as diretrizes iniciais do modelo. Para mitigar esta vulnerabilidade, a comunicação com a API ocorre estritamente do lado do servidor. O *backend* anexa, de forma programática e invisível para o cliente, um *System Prompt* rigoroso a cada pedido. Esta instrução restringe o escopo do assistente exclusivamente ao contexto do ISPGAYA e força a IA a ignorar perentoriamente quaisquer diretivas do utilizador que tentem alterar a sua configuração base.

Por fim, para evitar ataques de *Denial of Wallet* — onde submissões automatizadas em massa esgotam rapidamente os créditos da API — implementou-se um mecanismo de Limitação de Taxa (*Rate Limiting*). O servidor monitoriza os endereços IP de origem e impõe um limite estrito de pedidos por minuto para a rota do *chatbot*. Caso a quota seja excedida, a arquitetura interceta e rejeita os pedidos excedentes de imediato, devolvendo o código de estado HTTP 429 (*Too Many Requests*), salvaguardando a disponibilidade e o orçamento do Laboratório Cultural.

9. TESTES E VALIDAÇÃO

9.1 TESTES FUNCIONAIS

A fase de testes e validação constitui uma etapa basilar no ciclo de vida do desenvolvimento de *software*, assumindo a responsabilidade de atestar que o produto final cumpre integralmente os requisitos funcionais e não funcionais delineados nas fases de planeamento. O controlo de qualidade rigoroso não só permite a identificação precoce e a correção de defeitos (*bugs*), como também assegura a mitigação de riscos operacionais em ambiente de produção, garantindo uma experiência de utilização fluida e segura. Neste contexto, procedeu-se à execução manual e sistemática de um conjunto de cenários de teste críticos, mapeados para refletir as interações reais dos diferentes perfis de utilizadores (estudantes e administradores) com a plataforma do Laboratório Cultural. A Tabela 4 documenta a matriz de testes funcionais executados e os respetivos veredictos.

Tabela 4- Matriz de Testes Funcionais

ID do Teste	Funcionalidade Testada	Cenário de Teste	Resultado Esperado	Resultado Obtido	Evidência
TF 01	Autenticação no Back-office	Submissão de credenciais de administrador válidas no formulário de <i>login</i> .	Criação de sessão criptográfica e redirecionamento para o <i>dashboard</i> administrativo.	Passou	Acesso concedido e sessão listada no servidor.
TF 02	Controlo de Acessos (RBAC)	Tentativa de acesso direto a um URL protegido do <i>back-office</i> (ex: gestão de eventos) sem sessão ativa.	Interceção pelo decorador de segurança, bloqueio do acesso e redirecionamento para a página de <i>login</i> .	Passou	Redirecionamento com mensagem de erro "Acesso Negado".

TF 03	Validação de Criação de Conteúdo	Submissão de um novo evento no painel administrativo com campos obrigatórios (título e data) em branco.	Rejeição do pedido pelo servidor com sinalização dos campos em falta.	Passou	Formulário devolvido com mensagens de aviso nas <i>views</i> .
TF 04	Triagem de E-mails	Inscrição num evento submetendo um e-mail com domínio externo (ex: @gmail.com).	Rejeição do formulário pela regra de validação que exige o domínio @ispgaya.pt.	Passou	Erro de validação exibido no <i>front-end</i> .
TF 05	Prevenção de Duplicações	Nova tentativa de inscrição num espetáculo de teatro utilizando um e-mail já registado	Sistema deteta a concorrência na base de dados e impede a segunda inscrição.	Passou	Registo único na base de dados mantido.

		para esse evento.			
TF 06	Integridade do Mapa de Assentos	Seleção de um assento previamente reservado na grelha interativa do módulo de teatro.	O botão do assento apresenta-se com o estado "ocupado" (cor vermelha) e bloqueado a cliques.	Passou	Eventos de clique neutralizados via JavaScript.
TF 07	Consumo de Endpoints API	Efetuar um pedido GET à rota /api/v1/eventos a partir do navegador.	Retorno de um ficheiro JSON estruturado, contendo o <i>array</i> de eventos filtrados e código HTTP 200.	Passou	JSON válido validado em ferramenta de rede (<i>Network tab</i>).
TF 08	Envio de Notificações SMTP	Aprovação de uma requisição de livro no painel, desencadeando o	Geração de código de levantamento e receção da mensagem na caixa de correio	Passou	E-mail recebido com formatação HTML intacta.

		<i>pipeline</i> de e-mail.	institucional do aluno.		
--	--	-------------------------------	----------------------------	--	--

9.2 TESTES UNITÁRIOS E DE INTEGRAÇÃO (PYTEST)

Para atestar a estabilidade lógica do código *backend* de forma automatizada, recorreu-se à biblioteca `pytest`, uma *framework* de testes em Python reconhecida pela sua sintaxe expressiva e capacidade de asserção detalhada. Os testes unitários desempenham um papel fulcral no isolamento e na verificação do comportamento de funções específicas, enquanto os testes de integração garantem que os múltiplos componentes do ecossistema (como rotas, *controllers* e bases de dados) comunicam entre si sem anomalias.

O primeiro foco de teste automatizado incidiu sobre o módulo de segurança, mais concretamente na verificação do algoritmo de *hashing* `bcrypt`. Através de um teste unitário, comprovou-se que a natureza aleatória dos *salts* gerados pelo algoritmo produz *hashes* distintos mesmo quando a palavra-passe original é idêntica. Crucialmente, o teste assegura também que, independentemente da diferença na cadeia encriptada final, a função de verificação `checkpw` consegue atestar a validade de ambos os *hashes* face à *string* plana original, garantindo a robustez do sistema de autenticação contra ataques de dicionário e tabelas arco-íris (*rainbow tables*).

```
import pytest
import bcrypt
from app.utils.security import criar_hash_password, verificar_credenciais

def test_bcrypt_robustez_salts():
    """Valida se passwords iguais geram hashes diferentes, mas validam corretamente."""
    password_plana = "IspgayaCultural2024!"

    # Gerar dois hashes da mesma password
    hash_1 = criar_hash_password(password_plana)
    hash_2 = criar_hash_password(password_plana)

    # Asserção 1: Os hashes gerados devem ser obrigatoriamente diferentes
    assert hash_1 != hash_2, "Falha de segurança: Hashes idênticos para a mesma password."

    # Asserção 2: Ambos os hashes devem ser validados pela password plana
    assert verificar_credenciais(password_plana, hash_1) is True
    assert verificar_credenciais(password_plana, hash_2) is True
```

O segundo teste validou a consistência das Expressões Regulares (RegEx) aplicadas na triagem de utilizadores do *front-office*. Sendo a plataforma de uso académico, tornou-se imperativo atestar que o *backend* filtra de forma estrita o formato dos endereços de correio eletrónico submetidos. O teste unitário desenvolvido insere iterativamente uma bateria de e-mails válidos e inválidos na função de validação,

comprovando que apenas os endereços terminados estritamente em [@ispgaya.pt](mailto:ispgaya.pt) retornam um estado de aceitação.

```
from app.utils.validators import validar_email_institucional

def test_validacao_email_institucional():
    """Testa a triagem de domínios via RegEx."""
    emails_validos = [
        "aluno1234@ispgaya.pt",
        "docente.nome@ispgaya.pt"
    ]
    emails_invalidos = [
        "aluno@gmail.com",
        "docente@ispgaya.pt.com",
        "ispgaya.pt@aluno",
        "texto_aleatorio"
    ]

    for email in emails_validos:
        assert validar_email_institucional(email) is True

    for email in emails_invalidos:
        assert validar_email_institucional(email) is False
```

Para além das validações estruturais, a complexidade do módulo do Clube de Teatro exigiu a elaboração de um teste de integração focado na lógica transacional da bilheteira digital. O objetivo deste teste consistiu em validar o rigor matemático do controlo de lotação, garantindo que o *backend* rejeita liminarmente qualquer tentativa de *overbooking*. Através do *test client*, simulou-se a submissão de um formulário **POST** onde o utilizador tenta requisitar uma quantidade de bilhetes superior à capacidade restante do espetáculo. O teste confirma que o sistema deteta a insuficiência de vagas, aciona a reversão da transação na base de dados (*rollback*) e devolve o código HTTP 400 (*Bad Request*) acompanhado da respetiva mensagem de erro, atestando a eficácia do mecanismo de bloqueio pessimista previamente detalhado.

```
def test_integracao_reserva_lote_bilhetes(client):
    """Valida a transação de reserva e a proteção contra overbooking na bilheteira."""
    # Simulação de um pedido (POST) onde o utilizador tenta reservar 3 bilhetes
    # num espetáculo que (em ambiente de teste) apenas dispõe de 2 vagas restantes.
    payload_invalido = {
        "espetaculo_id": 1,
        "email_academico": "aluno.teste@ispgaya.pt",
        "quantidade_bilhetes": 3
    }

    resposta_invalida = client.post('/teatro/reservar', data=payload_invalido)

    # Asserção 1: O servidor deve rejeitar o pedido devido a falta de capacidade
    assert resposta_invalida.status_code == 400

    # Asserção 2: O sistema deve informar o utilizador da capacidade excedida
    # convertendo a resposta em texto legível para validação
    resposta_texto = resposta_invalida.data.decode('utf-8')
    assert "Lotação excedida" in resposta_texto or "Vagas insuficientes" in resposta_texto

    # Simulação de um pedido válido que esgota a lotação exata (2 bilhetes)
    payload_valido = payload_invalido.copy()
    payload_valido["quantidade_bilhetes"] = 2

    resposta_valida = client.post('/teatro/reservar', data=payload_valido)

    # Asserção 3: A transação válida deve ser processada com sucesso (HTTP 200 OK)
    assert resposta_valida.status_code == 200
```

Por fim, concebeu-se um teste de integração para avaliar o fornecimento de dados às interfaces assíncronas (AJAX). Através da funcionalidade de *test client* do Flask, injetou-se um pedido simulado ao *endpoint* da API responsável por listar as notícias. O teste verifica três parâmetros fundamentais de integração: a devolução do código de estado HTTP 200 (Sucesso), a tipologia da resposta (que deve ser obrigatoriamente um ficheiro JSON estruturado num *array*), e a presença das chaves estruturantes e metadados nos objetos iterados, atestando que a ligação entre o Model e o Controller opera devidamente.

```
def test_integracao_api_noticias(client):  
    """Valida a integridade e estrutura da rota RESTful de notícias."""  
    resposta = client.get('/api/v1/noticias')  
  
    # Valida código HTTP 200 OK  
    assert resposta.status_code == 200  
    assert resposta.is_json  
  
    dados = resposta.get_json()  
  
    # Valida se a resposta contém o formato esperado (array de objetos)  
    assert isinstance(dados, list)  
  
    # Valida a integridade das chaves estruturantes no primeiro objeto retornado  
    if len(dados) > 0:  
        noticia = dados[0]  
        assert "id" in noticia  
        assert "titulo" in noticia  
        assert "data_publicacao" in noticia
```

9.3 TESTES DE RESPONSABILIDADE

A diversidade de dispositivos utilizados para aceder ao Laboratório Cultural exigiu uma rigorosa bateria de testes de responsividade. No ambiente Desktop (testado na resolução padrão de 1920x1080 píxeis), a interface validou a sua estrutura de largura total, distribuindo a informação em grelhas (grids) de três colunas perfeitamente alinhadas, maximizando o espaço em branco para garantir a legibilidade. O menu de navegação apresentou-se no formato horizontal expansivo, tirando partido das classes utilitárias de visualização do Tailwind CSS para manter a coesão estética e o acesso rápido a todos os clubes.

No contexto de transição para Tablet (testado em 768x1024 píxeis, orientação vertical), o comportamento reativo do layout foi evidenciado pela redução automática da arquitetura visual de três para duas colunas. Os espaços marginais (padding) foram recalculados dinamicamente para acomodar o tamanho reduzido do ecrã sem causar sobreposição de texto ou colapso dos cartões de eventos. As imagens e tipografias escalaram proporcionalmente, comprovando a robustez dos pontos de quebra (breakpoints) estipulados no código CSS.

O cenário mais exigente centrou-se no ambiente Mobile (375x667 píxeis). Verificou-se a contração bem-sucedida do layout para uma coluna única e a substituição da barra de navegação por um menu embutido colapsável (ícone "hambúrguer"). Especial atenção foi dedicada ao mapa de lugares da bilheteira do teatro, cuja adaptação impunha um desafio técnico de usabilidade. A solução implementada recorreu ao controlo de transbordo da grelha (overflow-x-auto), permitindo que a planta da sala mantenha as suas proporções originais através de um scroll horizontal dedicado. Esta abordagem garantiu que os botões dos assentos mantivessem o seu tamanho tátil mínimo (alvo de toque), prevenindo cliques acidentais e frustração na interação em ecrãs táteis.

9.4 VALIDAÇÃO DE FORMULÁRIOS

A validação do processamento de dados submetidos na plataforma foi testada sob uma perspectiva de dupla camada, combinando verificações do lado do cliente e do lado do servidor. A primeira camada, implementada via JavaScript no *front-end*, atuou com eficácia na interceção de dados inválidos (como a formatação incorreta de e-mails ou a omissão de campos obrigatórios) antes mesmo de o pedido ser enviado ao servidor. Esta mecânica provou ser crucial para a redução de latência e poupança de recursos de rede, fornecendo um *feedback* visual imediato aos utilizadores através da alteração das bordas dos campos para tons de alerta e da exibição de mensagens dinâmicas sob as caixas de texto.

Contudo, a validação no cliente não substitui o rigor do controlo no servidor, visto poder ser facilmente contornada por utilizadores mal-intencionados através da manipulação do DOM ou do envio de pedidos HTTP diretos. Assim, testou-se exaustivamente a segunda camada de validação desenvolvida em Python. Verificou-se que as rotas do *backend* inspecionam de forma independente todos os *payloads* recebidos, revalidando extensões de ficheiros, tamanhos de *uploads* e integridade referencial. Esta arquitetura atestou que a camada do servidor funciona como uma barreira final intransponível, processando transações unicamente quando todos os critérios de negócio e segurança são irrepreensivelmente cumpridos.

9.5 COMPATIBILIDADE DE MOTORES DE RENDERIZAÇÃO (BROWSERS)

Dado o carácter público da plataforma, a homologação de compatibilidade transversal a diferentes navegadores (*Cross-Browser Testing*) revelou-se indispensável. Os testes incidiram sobre os três principais motores de renderização que dominam o mercado: o motor Blink (suportando o Google Chrome, Edge e Opera), o motor Gecko (suportando o Mozilla Firefox) e o motor WebKit (nativo do Safari no ecossistema macOS e iOS). O objetivo central foi assegurar uma perceção e operabilidade idênticas, mitigando comportamentos discrepantes originados por diferentes interpretações de marcação web e rotinas de JavaScript.

Os resultados obtidos atestaram uma estabilidade notável em todos os motores. A renderização do CSS suportado por Flexbox e Grid Layout comportou-se de forma uniforme, não se registando quebras de componentes. As rotinas assíncronas de JavaScript e a manipulação do DOM, incluindo ferramentas de terceiros para gráficos de estatísticas do back-office (como o Chart.js), processaram dados fluida e instantaneamente em qualquer browser. Adicionalmente, verificou-se o correto

funcionamento do localStorage, atestando que o consentimento no modal de cookies se mantém persistente e é lido uniformemente por todos os motores, garantindo conformidade legal e uma experiência contínua aos visitantes.

10. CONCLUSÃO

10.1 BALANÇO DO PROJETO E COBERTURA DE REQUISITOS

O desenvolvimento da plataforma "Laboratório Cultural" para o Instituto Superior Politécnico Gaya (ISPGAYA) culminou num produto de software funcional, robusto e perfeitamente alinhado com os propósitos pedagógicos e operacionais delineados na sua fase de conceptualização. A integração transversal de conhecimentos das unidades curriculares de Aplicações e Tecnologias Web (ATW) e Linguagens e Paradigmas da Programação (LPP) permitiu a construção de um ecossistema digital que centraliza e democratiza a oferta cultural da instituição. O balanço final revela que a arquitetura adotada, sustentada na framework Flask e no padrão MVC, revelou-se adequada para gerir a complexidade transaccional exigida pelos vários módulos da plataforma.

A validação do projeto atesta que a transição de um modelo de gestão cultural analógico para um paradigma digital e desmaterializado foi concretizada com sucesso. A implementação rigorosa de mecanismos de engenharia de software garantiu não apenas a operacionalidade de funcionalidades complexas, como a bilheteira interativa e o catálogo literário, mas também a adoção de boas práticas de segurança, design responsivo e inovação tecnológica, evidenciada pela integração de Inteligência Artificial.

De forma a materializar a avaliação do sucesso do projeto com o devido rigor técnico e transparência, a Tabela 5 apresenta a matriz de cobertura dos requisitos funcionais e técnicos. Para o efeito, distingue-se aquilo que consubstancia um produto pronto para produção (Totalmente Implementado) daquilo que, devido a restrições de escopo académico, se mantém numa fase de Produto Mínimo Viável (Parcialmente Implementado) ou de validação de conceito (Totalmente Implementado).

Tabela 5- Matriz de Cobertura de Requisitos e Estado Real

ID	Requisito / Módulo	Estado Real	Estratégia e Notas Técnicas
RF01	Portal ISPGAYA	Totalmente Implementado	Reprodução da identidade visual com Tailwind CSS, paleta corporativa e banner de consentimento de cookies.
RF02	Biblioteca / Leitura	Parcialmente Implementado	Catálogo literário e submissões operacionais (MVP). A gestão de devoluções requer validação manual no painel.
RF03	Bilheteira de Teatro	Totalmente Implementado	Módulo de controlo de concorrência com bloqueio pessimista no backend, validando matematicamente a lotação.
RF04	Módulo da Tuna	Totalmente Implementado	Listagem de elementos e integração de galeria dinâmica com interatividade em JavaScript.
RF05	Filtros de Eventos	Totalmente Implementado	Implementação de pesquisa assíncrona (AJAX) para filtrar agenda cultural externa sem recarregar a página.
RF06	Módulo de Notícias	Totalmente Implementado	Mapeamento e listagem cronológica de artigos institucionais com suporte multimédia.
RF07	Painel de Back-office	Totalmente Implementado	Interface administrativa protegida para operações CRUD sobre todos os recursos culturais da plataforma.
RF08	Tickets em PDF	Totalmente Implementado	Uso da biblioteca fpdf2 para geração de bilhetes, integração de códigos QR e expedição por correio (SMTP).

RF09	Chatbot Inteligência Artificial	Protótipo Funcional	Integração da API Gemini via chamadas AJAX. Opera como prova de conceito (PoC), sujeito a Rate Limits.
RNF01	Segurança	Totalmente Implementado	Imposição de controlo de acessos (RBAC), parametrização SQL (SQLi), sanitização (XSS) e encriptação bcrypt.

10.2 MÉTRICAS DE ENGENHARIA DE SOFTWARE

A dimensão e a complexidade do sistema desenvolvido podem ser quantificadas através de métricas padrão de engenharia de *software*, que refletem o volume de trabalho e o nível de abstração alcançado. A plataforma foi arquitetada com uma clara divisão de responsabilidades, resultando numa base de código extensa, porém modular, que facilita a legibilidade e futuras manutenções. O equilíbrio entre o código lógico de servidor e o código de apresentação atesta o cuidado dedicado tanto à performance computacional como à riqueza da interface de utilizador.

A estruturação rigorosa em *Blueprints* e a normalização de dados refletem-se diretamente nos indicadores do projeto. A Tabela 6 sistematiza o volume de desenvolvimento através das métricas mais relevantes da plataforma.

Tabela 6 - Métricas do Projeto*Tabela 6- Métricas do Projeto*

Métrica	Quantidade	Detalhe Estrutural
Linhas de Código (LOC)	~5.300	Cerca de 2.100 linhas em lógica pura Python (<i>backend</i>) e 3.200 linhas em <i>templates</i> de interface Jinja2, englobando HTML5, Tailwind CSS e Vanilla JS (<i>frontend</i>).
Ficheiros Modulares	48	Organização em pacotes estruturados compreendendo <i>views</i> , <i>controllers</i> , modelos, utilitários de segurança e módulos de configuração.
Endpoints (Rotas)	37	Rotas independentes distribuídas por diferentes <i>Blueprints</i> (frontoffice, back-office, auth, api), gerindo renderização de páginas e respostas JSON.
Tabelas Relacionais	12	Entidades da base de dados normalizadas até à 3ª Forma Normal (3NF), garantindo integridade referencial através de chaves primárias e estrangeiras.

10.3 DIFICULDADES ENCONTRADAS E RESOLUÇÃO DE PROBLEMAS

O ciclo de desenvolvimento do Laboratório Cultural não esteve isento de desafios técnicos, exigindo da equipa de desenvolvimento uma capacidade constante de pesquisa e adaptação metodológica. O primeiro obstáculo de relevo prendeu-se com a necessidade de alinhar o sistema com a identidade corporativa do portal principal do ISPGAYA. Face à indisponibilidade de um *Design System* formal e documentado, procedeu-se a um processo de engenharia reversa (*reverse engineering*) utilizando as *Developer Tools* (DevTools) dos navegadores. Este esforço permitiu extrair códigos hexadecimais de cores, regras tipográficas e espaçamentos originais, transpondo-os com sucesso para a configuração global do Tailwind CSS.

Ao nível da persistência de dados, a implementação de uma arquitetura híbrida gerou constrangimentos sintáticos indesejados. O motor SQLite (usado em desenvolvimento) e o MySQL (usado em produção) exigem formatos de parametrização SQL distintos para a prevenção de injeções (`?` no primeiro, `%s` no segundo). A tentativa de partilhar o mesmo código para ambos os ambientes originou erros de execução. A solução técnica passou pelo desenvolvimento de um *wrapper* de acesso a dados (`SQLiteCursorWrapper`) que interceta as *queries* em tempo de execução e traduz os *placeholders* ativamente com base no dialeto ativado pelas variáveis de ambiente, garantindo portabilidade absoluta.

Por fim, a geração automatizada de bilhetes exigiu uma otimização criteriosa de recursos do servidor. O *pipeline* inicial de criação de PDFs embebidos com códigos QR requeria a gravação temporária de imagens no disco físico, o que introduzia uma latência inaceitável na resposta HTTP (I/O bloqueante). O obstáculo foi superado através da alteração do paradigma de processamento: os códigos QR e os documentos PDF passaram a ser gerados inteiramente na memória RAM como fluxos de *bytes* (`BytesIO`), sendo enviados diretamente pelo serviço SMTP e descartados em seguida, resultando num tempo de resposta imediato para o utilizador.

10.4 TRABALHO FUTURO

Considerando que o *software* é um produto em constante evolução, perspetivam-se diversas linhas de desenvolvimento técnico para uma futura iteração da plataforma. A nível de infraestrutura e operações de desenvolvimento (*DevOps*), recomenda-se a transição para um ambiente de produção suportado por contentorização, utilizando tecnologias como o Docker. Esta abordagem, aliada à implementação de *pipelines* de Integração Contínua e Entrega Contínua (CI/CD), garantirá a automação de testes e a estabilidade das atualizações em ambientes reais de alojamento na *cloud*, eliminando as discrepâncias entre os ambientes de desenvolvimento e produção.

No domínio da escalabilidade e *performance* arquitetónica, a gestão de processos constitui um vetor de melhoria prioritário. Atualmente, a execução de tarefas morosas — como a geração de bilhetes em formato PDF e o respetivo envio via protocolo SMTP — ocorre no mesmo ciclo de processamento da resposta HTTP. A introdução de uma fila de mensagens e gestão de tarefas em segundo plano (*background workers*), suportada por tecnologias da indústria como o Celery e o Redis, permitirá delegar estas operações assíncronas de forma não bloqueante, otimizando drasticamente a disponibilidade do servidor principal e reduzindo a latência percebida pelo utilizador.

Relativamente à garantia de qualidade (*Quality Assurance*), e de forma a complementar a base de testes unitários e de integração já estabelecida com o *pytest*, projeta-se a inclusão de testes *End-to-End* (E2E) automatizados. A adoção de ferramentas de orquestração de navegadores, como o Selenium, permitirá simular o comportamento de navegação real do utilizador. Esta técnica revelar-se-á crucial para homologar o correto funcionamento das interações visuais e do controlo de concorrência na grelha de assentos da bilheteira de teatro de forma contínua.

Por fim, do ponto de vista funcional, planeia-se a integração de APIs externas (como a Google Calendar API) para a sincronização automatizada da programação cultural. Em sinergia, perspetiva-se a evolução do *front-office* para uma *Progressive Web App* (PWA) dotada de suporte multilíngue (*i18n*). Estas características conferirão à plataforma capacidades de execução *offline* robustas e garantirão a inclusão plena de estudantes oriundos de programas de mobilidade internacional, consolidando o Laboratório Cultural como uma referência tecnológica e social no ISPGAYA.

10.5 VISÃO CRÍTICA E OPORTUNIDADES DE MELHORIA ARQUITETÓNICA

Uma análise introspectiva ao código e à arquitetura adotada revela oportunidades significativas de aproximação aos padrões atuais estabelecidos pela indústria da engenharia de *software*. Em primeiro lugar, embora o desenvolvimento de interrogações SQL cruas e parametrizações manuais tenha cumprido rigorosamente os objetivos pedagógicos da disciplina de LPP, a transição para um *Object-Relational Mapper* (ORM) maduro, como o SQLAlchemy, é fortemente aconselhada. O uso de um ORM permitiria abstrair por completo a complexidade sintática dos diferentes motores de bases de dados, reduziria o código transacional ou repetitivo (DRY) e forneceria mecanismos mais elegantes e seguros para gestão de migrações estruturais (*schema migrations*).

Em segundo lugar, a camada de segurança e gestão de estado pode ser modernizada. O sistema atual depende de uma arquitetura baseada em sessões de servidor (*stateful*) controladas por *cookies*, adequada para aplicações web tradicionais. Contudo, perante o crescimento das rotas RESTful integradas no *blueprint* da API, a migração para a autenticação baseada em *JSON Web Tokens* (JWT) revelar-se-ia superior. O paradigma *stateless* proporcionado pelos JWT libertaria o servidor do armazenamento de dados de sessão, facilitando horizontalmente a escalabilidade do sistema e preparando a infraestrutura para o consumo por clientes *mobile* nativos independentes.

Por fim, a nível de arquitetura de dados, identifica-se espaço para otimização da normalização estrutural. No modelo atual, existem tabelas distintas e com redundância conceptual para acomodar entidades ligeiramente diferentes, como membros da Tuna e membros de outras iniciativas. Numa abordagem avançada, estas tabelas poderiam ser unificadas numa única entidade genérica parametrizada (por exemplo, *membros_clubes*), gerida por chaves estrangeiras que definiriam dinamicamente as suas afiliações a diferentes valências do projeto. Esta refatorização simplificaria os relacionamentos da base de dados e tornaria o sistema mais flexível à eventual criação de novos clubes culturais no futuro.

10.6 REFLEXÃO FINAL E APRENDIZAGENS

A elaboração deste projeto final de curso providenciou um ambiente de excelência para a consolidação e aplicação prática de vastos conceitos teóricos de engenharia informática. Uma das aprendizagens mais valiosas incidiu sobre o tratamento da gestão de concorrência em sistemas transacionais web. O desenvolvimento prático da bilheteira expôs a necessidade imperiosa de salvaguardar a integridade de dados através de validações atómicas, demonstrando como falhas logísticas podem ocorrer se múltiplos utilizadores tentarem requisitar o mesmo recurso simultaneamente (o clássico problema do *overbooking*), e como os bloqueios transacionais constituem a resposta de engenharia adequada.

Adicionalmente, a equipa adquiriu competências vitais na gestão de estruturas de dados incompatíveis de forma nativa com as restrições dos motores relacionais clássicos. O problema da gravação da matriz de lugares seleccionados (arrays/listas) numa tabela que apenas aceita tipos de dados escalares exigiu a aplicação de mecanismos de serialização JSON e processamento imperativo em Python. Esta técnica ilustrou como diferentes paradigmas de programação (como o processamento de listas funcionais) podem ser hibridizados para contornar limitações arquitetónicas sem comprometer a eficiência ou a 3ª Forma Normal da base de dados.

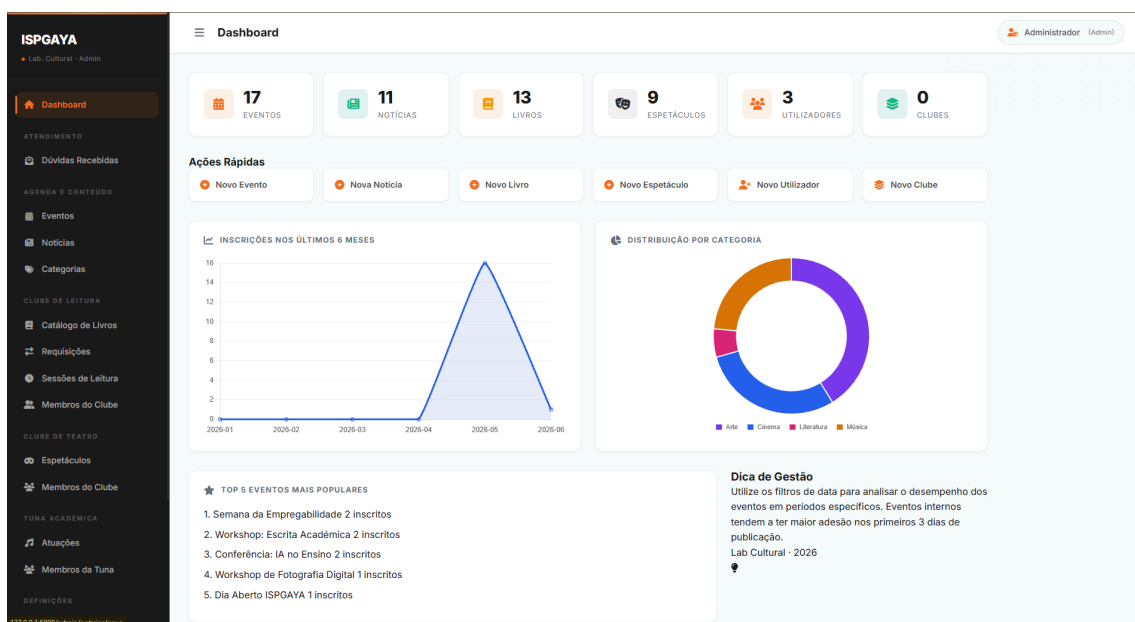
Em suma, a construção do "Laboratório Cultural" exigiu uma orquestração rigorosa entre o controlo de fluxos, a interface visual e a segurança de rede, culminando na compreensão profunda do ciclo de expiração de dados em *cache* e na proteção de sessões ativas. O projeto transcende a mera aplicação de linguagens de programação; evidencia a transformação de uma especificação teórica num produto resiliente, espelhando a maturidade técnica da equipa face aos desafios da arquitetura *web* moderna.

ANEXO A — GUIÃO DE EVIDÊNCIAS VISUAIS DE TESTES FUNCIONAIS

De modo a documentar e atestar a correta execução da matriz de testes funcionais descrita no Capítulo 9 (Tabela 4), compila-se o presente roteiro de evidências visuais. As capturas de ecrã a seguir apresentadas ilustram o comportamento reativo e lógico do sistema face a diferentes cenários de utilização e validação.

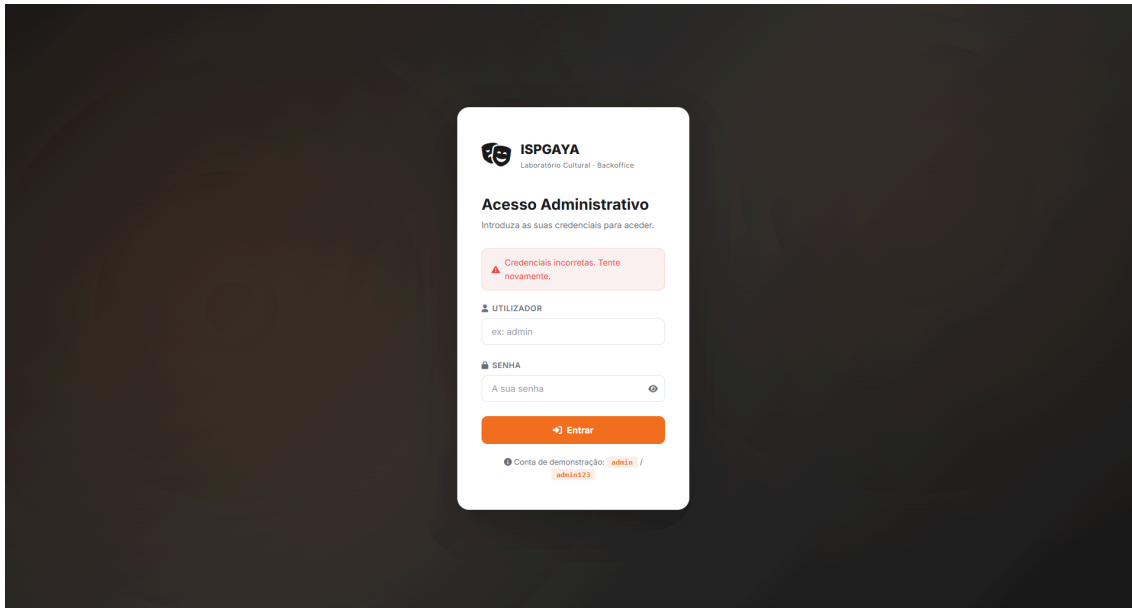
ANEXO 1: AUTENTICAÇÃO VÁLIDA

A Figura A1 apresenta o painel do Dashboard Administrativo após a realização de um login com sucesso. Esta evidência comprova que o sistema gera corretamente a sessão criptográfica e redireciona o utilizador autenticado para a interface de gestão, desbloqueando o acesso aos módulos de controlo (CRUD).



ANEXO 2: PROTEÇÃO DE ROTAS

A Figura A2 demonstra a mensagem de erro e o respetivo redirecionamento automático aquando da tentativa de acesso direto a um URL administrativo por parte de um utilizador sem sessão ativa. Comprova-se, assim, a eficácia do decorador `@login_required` na interceção e bloqueio de requisições HTTP não autorizadas.



ANEXO 3: VALIDAÇÃO DE FORMULÁRIOS

A Figura A3 ilustra os alertas visuais de erro submetidos no formulário de criação de eventos. Através desta captura, evidencia-se a dupla validação do sistema, que impede o registo de entradas com campos obrigatórios vazios, garantindo a integridade dos dados antes de qualquer transação na base de dados.

Criar Novo Evento
Preencha os dados para adicionar um novo evento à agenda.

O campo "titulo" é obrigatório. O campo "data_evento" é obrigatório. O campo "descricao" é obrigatório. O campo "hora" é obrigatório. O campo "categoria_id" é obrigatório.

TÍTULO *
Ex: Noite de Teatro

DESCRIÇÃO *
Descreva o evento...

DATA E TIPO

DATA * dd.mm.aaaa

HORA * --:--

TIPO * Interno (ISPGAYA)

☐ AGENDAR PUBLICAÇÃO PARA UMA DATA ESPECÍFICA?

ANEXO 4: RESTRIÇÃO DE E-MAIL ACADÉMICO

A Figura A4 documenta a mensagem de erro acionada ao tentar efetuar uma requisição utilizando um e-mail pessoal não académico. Fica comprovado que as Expressões Regulares (RegEx) implementadas no backend filtram e rejeitam eficazmente qualquer domínio que não corresponda estritamente ao formato @ispgaya.pt.

The screenshot shows a web form titled "Junta-te ao Clube de Teatro" in orange text. Below the title, a light gray box contains the following elements:

- Text: "Tens paixão pela representação ou queres desenvolver a tua presença e autoconfiança? Preenche o formulário para te juntares aos nossos ensaios semanais!"
- Form field for "Nome Completo:" with the value "João Baião".
- Form field for "E-mail Institucional:" with the value "joao12@gmail.com". This field is outlined with a red border.
- Red error message: "▲ Apenas e-mails @ispgaya.pt são permitidos."
- Two buttons: "Submeter Inscrição" (orange) and "Cancelar" (gray).

ANEXO 5: CONTROLO DE DUPLICAÇÃO

A Figura A5 exibe o alerta de "inscrição duplicada" devolvido pelo módulo do Clube de Teatro. Confirma-se que a base de dados deteta a tentativa de redundância transacional para o mesmo endereço de e-mail num espetáculo específico, prevenindo infrações às regras de lotação.

The screenshot displays a web interface for joining a theater club. At the top, the title "Junta-te ao Clube de Teatro" is underlined in orange. Below the title, a red error message box states: "Já te inscreveste no Clube de Teatro com este e-mail!". The main text of the form asks if the user is passionate about theater or wants to develop their presence and self-confidence, and prompts them to fill out the form to join weekly rehearsals. There are two input fields: "Nome Completo:" with the placeholder "O teu nome completo" and "E-mail Institucional:" with the placeholder "aluno@ispgaya.pt". At the bottom, there are two buttons: "Submeter Inscrição" in orange and "Cancelar" in gray.

ANEXO 6: MAPA DE SALA

A Figura A6 apresenta a grelha interativa de lugares do teatro. A evidência atesta o processamento dinâmico efetuado pelo JavaScript e a serialização de dados em Python, traduzindo os lugares ocupados na base de dados em botões com estado visual indisponível (a vermelho) e inoperáveis para o cliente atual.

Reservar Lugar — Workshop: Presença de Palco e Dicção

Para reservar o seu lugar no espetáculo, por favor identifique-se com os seus dados institucionais. Os campos marcados com * são obrigatórios.

Selecione os seus lugares

PALCO

A	1	2	3	4	5	6	7	8	9	10	11	12
B	1	2	3	4	5	6	7	8	9	10	11	12
C	1	2	3	4	5	6	7	8	9	10	11	12
D	1	2	3	4	5	6	7	8	9	10	11	12
E	1	2	3	4	5	6	7	8	9	10	11	12
F	1	2	3	4	5	6	7	8	9	10	11	12
G	1	2	3	4	5	6	7	8	9	10	11	12
H	1	2	3	4	5	6	7	8	9	10	11	12

☐ Livre ☒ Selecionado ☐ Ocupado

Detalhes da Sessão

03/06/2026
14:00
ISPGAYA
Gratuita

Dúvidas sobre a sua reserva ou bilhete?
Contacte a produção:
[Produção Teatro](#)

ANEXO 7: PAYLOAD JSON DA API

A Figura A7 ilustra a visualização do retorno de um endpoint RESTful da API no navegador. Comprova-se que a arquitetura Flask serializa e devolve as estruturas de dados de forma correta no formato de um array de objetos JSON, confirmando a viabilidade do consumo de dados assíncrono para o front-end.

```
{
  "eventos": [
    {
      "ativo": 1,
      "atualizado_em": "2026-05-13T21:38:02",
      "categoria_cor": "#16a34a",
      "categoria_icone": "\ud83d\udcda",
      "categoria_id": 3,
      "categoria_nome": "Literatura",
      "cidade": "Vila Nova de Gaia",
      "criado_em": "2026-05-13T21:38:02",
      "data_evento": "2026-06-02",
      "data_publicacao": null,
      "descricao": "Aprende as melhores pr\u00e1ticas para redigir o teu trabalho de fim de curso ou teses.",
      "destaque": 0,
      "hora": "17:30",
      "id": 7,
      "imagem": "img/eventos/evento_7.jpg",
      "inscritos": 2,
      "limite_bilhetes": 30,
      "link_bilheteira": null,
      "link_externo": null,
      "local": "Biblioteca ISPGAYA",
      "local_capacidade": 50,
      "local_id": 2,
      "local_morada": "Av. dos Descobrimentos, 333",
      "preco": 0,
      "tipo": "Interno",
      "tipo_ingresso": "gratuito",
      "titulo": "Workshop: Escrita Acad\u00e9mica"
    },
    {
      "ativo": 1,
      "atualizado_em": "2026-05-13T21:38:02",
      "categoria_cor": "#2563eb",
      "categoria_icone": "\ud83c\udfac",
      "categoria_id": 5,
      "categoria_nome": "Cinema",
      "cidade": "Vila Nova de Gaia",
      "criado_em": "2026-06-13T21:38:02",
      "data_evento": "2026-06-12",
      "data_publicacao": null,
      "descricao": "24 horas para desenvolver solu\u00e7\u00f5es tecnol\u00f3gicas para melhorar a vida acad\u00e9mica no ISPGAYA.",
      "destaque": 0,
      "hora": "10:00",
      "id": 5,
      "imagem": "img/eventos/evento_5.jpg",
      "inscritos": 0,
      "limite_bilhetes": 50,
      "link_bilheteira": null,
      "link_externo": null,
      "local": "Sala de Atos",
      "local_capacidade": 100,
      "local_id": 3,
      "local_morada": "Edif\u00e9cio Central",
      "preco": 0,
      "tipo": "Interno",
      "tipo_ingresso": "gratuito",
      "titulo": "Hackathon: Solu\u00e7\u00f5es Campus"
    },
    {
      "ativo": 1,
      "atualizado_em": "2026-05-13T21:38:02",
      "categoria_cor": "#9333ea",
      "categoria_icone": "\ud83c\udfa8",
      "categoria_id": 4,
      "categoria_nome": "Arte",
      "cidade": "Vila Nova de Gaia",
      "criado_em": "2026-06-13T21:38:02",
      "data_evento": "2026-06-18",
      "data_publicacao": null,
      "descricao": "Desafio de inten\u00e7\u00e3o com especialistas de cada esfera para criar solu\u00e7\u00f5es de modo a cada esfera."
```

ANEXO 8: NOTIFICAÇÃO SMTP

A Figura A8 exibe a receção de um correio eletrónico de confirmação gerado e enviado pela plataforma. Esta captura consubstancia a correta configuração do protocolo SMTP, comprovando que o servidor emite automaticamente os bilhetes eletrónicos (PDF) ou códigos de requisição de forma assíncrona.



Reservas do Espetáculo

Administrador

(Admin)

Gestão de Lugares: Falar Verdade a Ler (Almeida Garrett)

Voltar

Mapa de Ocupação

PALCO

A

1

2

3

4

5

6

7

8

9

10

11

12

B

1

2

3

4

5

6

7

8

9

10

11

12

C

1

2

3

4

5

6

7

8

9

10

11

12

D

1

2

3

4

5

6

7

8

9

10

11

12

E

1

2

3

4

5

6

7

8

9

10

11

12

F

1

2

3

4

5

6

7

8

9

10

11

12

G

1

2

3

4

5

6

7

8

9

10

11

12

H

1

2

3

4

5

6

7

8

9

10

11

12

Libre

Ocupado